Universidad de Ingeniería y Tecnología · Maestría
Taller de Arquitectura Big Data · Ciclo 2026-2

Lumina Bank

Plataforma de datos batch y streaming en Google Cloud

Entrega 2: procesamiento, transformación y modelado

Bryan Montoya
David Choque
Henry Godiño

Lima, 11 de noviembre de 2026

Índice

1. Qué cambiamos después de la Entrega 1 **2**

2. Arquitectura en notación C4 **2**

2.1. Nivel 1: contexto . . . 2
2.2. Nivel 2: camino streaming . . . 3
2.3. Nivel 2: camino batch, modelado y exposición . . . 3
2.4. Conexiones entre componentes . . . 3
2.5. Capas de gobierno, seguridad y redes . . . 4

3. Por qué elegimos cada componente **9**

4. Dimensionamiento de Dataflow y Cloud Run **11**

4.1. Volumen de referencia . . . 11
4.2. Dataflow: lo que medimos . . . 11
4.3. Cloud Run . . . 14
4.4. Impacto en el costo . . . 14

5. Implementación de la Entrega 2 **15**

5.1. Pipeline batch con Spark . . . 15
5.2. Pipeline streaming con Apache Beam . . . 16
5.3. Batch y streaming en el mismo modelo . . . 16
5.4. Modelo estrella en Gold . . . 17
5.5. Modelos en BigQuery ML . . . 17
5.6. API de datos . . . 18

6. Evidencias de la implementación **18**

6.1. Proyecto e infraestructura base . . . 19
6.2. Gobierno, seguridad y redes . . . 21
6.3. Pipeline batch . . . 25
6.4. Pipeline streaming . . . 28
6.5. Pruebas de carga . . . 30
6.6. Gold y BigQuery ML . . . 33
6.7. API de datos en Cloud Run . . . 36

7. Pendientes para la Entrega 3 **38**

A. Cambios en el repositorio **40**

1 Qué cambiamos después de la Entrega 1

La revisión de la primera entrega nos dejó cuatro observaciones. Las atendimos todas y, además, desplegamos en un proyecto de Google Cloud (`lumina-bank-utec`) los caminos batch y streaming, la capa Gold, los modelos de BigQuery ML, la API y buena parte de las capas transversales, para medir en lugar de suponer. No es la plataforma completa: la tabla 3 indica qué quedó fuera y por qué.

El laboratorio corre en `us-central1` y en la multirregión US por costo y porque ahí están disponibles todos los servicios que usamos. El diseño sigue siendo el de la Entrega 1: región principal en Santiago (`southamerica-west1`) y continuidad en São Paulo. Por eso los costos se calculan con precios de `us-central1` y luego se ajustan a Santiago (sección 4.4). La tabla 1 resume cada cambio.

Tabla 1. Observaciones de la docente y respuesta del equipo

Observación	Qué hicimos	Sección
Mostrar las conexiones entre componentes	Rehicimos la arquitectura en notación C4 con LikeC4: una vista de contexto, una por cada camino de datos y una por cada capa transversal. Cada flecha dice qué dato viaja y por qué canal. Numeramos 18 conexiones y las describimos en una tabla.	2
Incluir gobierno de datos, seguridad y redes	Las tres capas tienen su propia vista C4. En el laboratorio desplegamos la VPC privada, la llave KMS con rotación, la taxonomía de policy tags, el escaneo de calidad, el linaje, las vistas por entidad, la auditoría y Dataflow sin IP pública. Falta asignar los policy tags a las columnas y llevar Dataproc a la VPC privada. VPC Service Controls, Cloud VPN y el balanceador con Cloud Armor quedan diseñados; la tabla 3 dice por qué.	2.5
Cloud Run sobredimensionado	Lo recalculamos con la ley de Little sobre el volumen real. El servicio de autorización pasa de 2–100 a 2–20 instancias y la API de datos, de 2–100 a 0–3.	4.3
Dimensionar y validar Dataflow	Hicimos tres pruebas de carga con un worker fijo. La capacidad real resultó muy por debajo de lo que suponíamos; con ese dato fijamos el rango de workers de la fase MVP y una meta de optimización para producción.	4.2
Justificar cada componente	Para cada servicio indicamos qué hace en la solución, por qué lo elegimos y qué alternativa descartamos.	3

2 Arquitectura en notación C4

Usamos el modelo C4 porque separa los niveles de detalle: primero quién usa la plataforma y con qué sistemas se conecta (contexto) y después qué piezas la componen (contenedores). El modelo está escrito en LikeC4 (`doc/arquitectura/lumina.c4`), así que el diagrama se versiona junto con el código y se regenera con un comando. Cada componente lleva el ícono oficial del servicio de Google Cloud que representa.

2.1 Nivel 1: contexto

La plataforma recibe transacciones de los canales de las 12 entidades, cambios de saldos de sus cores bancarios, telemetría de cajeros y datos públicos. Entrega alertas al equipo antifraude, tableros a la gerencia y a cada entidad sus propios datos junto con la señal de fraude compartida.

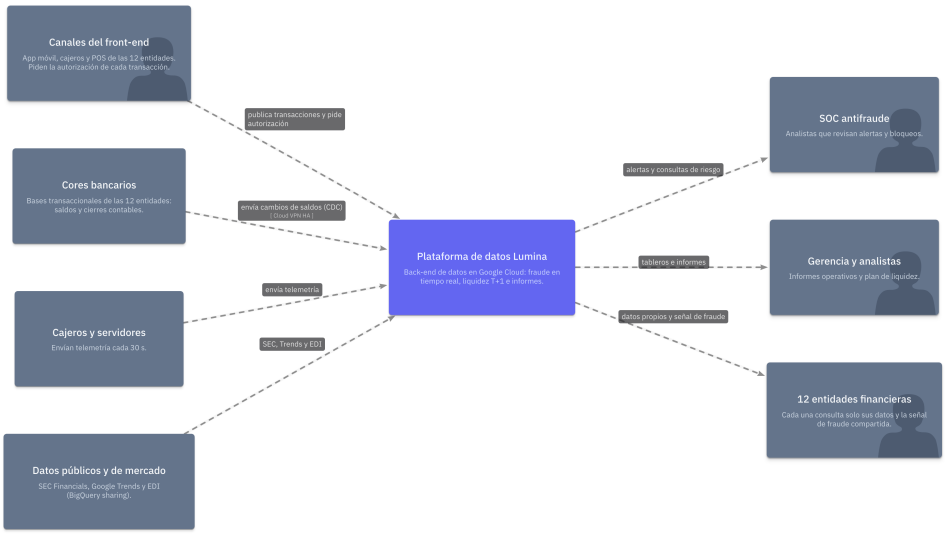


Figura 1. Diagrama de contexto (C4, nivel 1)

2.2 Nivel 2: camino streaming

Es el camino del fraude en tiempo real. La transacción entra por Pub/Sub, Dataflow la evalúa con estado por cliente y por destino, deja las *features* en Memorystore para el servicio de autorización y escribe el resultado en BigQuery. Las alertas salen por otro tópico hacia el SOC. Una copia cruda de cada evento queda en el Data Lake para auditoría y reproceso. El diagrama está en la figura 2 (página 5).

2.3 Nivel 2: camino batch, modelado y exposición

El histórico y los cierres llegan al Data Lake por Datastream y Storage Transfer. Spark en Dataproc Serverless los limpia y los deja en Parquet; de ahí pasan a Silver y, con SQL, a Gold. BigQuery ML entrena sobre Gold y devuelve el pronóstico de liquidez. Managed Service for Apache Airflow (antes Cloud Composer) orquesta el proceso diario (figura 3, página 6).

2.4 Conexiones entre componentes

La numeración coincide con la de los diagramas.

Tabla 2. Conexiones de la arquitectura

N.°	Origen → destino	Qué viaja	Canal
1	Canales → Pub/Sub `tx-eventos`	Evento de transacción en JSON	gRPC con TLS; esquema en el tópico solo diseñado
2	Pub/Sub → Dataflow	El mismo evento	Suscripción *pull*, deduplicación por `event_id`
3	Dataflow → Memorystore	Último score, ubicación y conteo de 24 h por cliente	Redis dentro de la VPC
4	Dataflow → BigQuery Silver	Transacción con score, decisión y latencia	Storage Write API
5	Dataflow → Pub/Sub `alertas-fraude`	Transacciones REVISAR o BLOQUEAR	Publicación en Pub/Sub

N.°	Origen → destino	Qué viaja	Canal
6	Pub/Sub → Data Lake	Evento crudo	Suscripción a Cloud Storage cada 5 min
7	Terminales → Pub/Sub → Bronze	Telemetría	Suscripción BigQuery, sin código
8	Cores → Datastream → Data Lake	Cambios de saldos y cierres (CDC)	Cloud VPN de alta disponibilidad
9	Data Lake ↔ Dataproc	CSV crudo de entrada, Parquet curado de salida	Conector de GCS, workers sin IP pública. En el laboratorio el batch corrió en la VPC `default`
10	Data Lake → Silver	`transacciones_batch`	`bq load` de Parquet
11	Datos públicos → BigQuery SQL	SEC, Google Trends y EDI	Consulta directa y BigQuery sharing
12	Silver y Bronze → SQL → Gold	Dimensiones y hechos	SQL dentro de BigQuery
13	Gold → BigQuery ML	Series de efectivo y transacciones etiquetadas	`CREATE MODEL`, `ML.FORECAST`
14	Memorystore → autorización	*Features* del cliente	Direct VPC egress
15	Silver y Gold → API de datos	Alertas, riesgo, liquidez y KPI	Consultas parametrizadas
16	Gold → Looker Studio	Vistas `v_kpi_*`	Conector de BigQuery con BI Engine
17	Gold → 12 entidades	Datos propios y señal de fraude	Vistas autorizadas y filtro por entidad
18	Pub/Sub → SOC	Alertas	Suscripción *push*

2.5 Capas de gobierno, seguridad y redes

Cada capa tiene su vista. Arriba están los controles y abajo, los componentes sobre los que actúan.

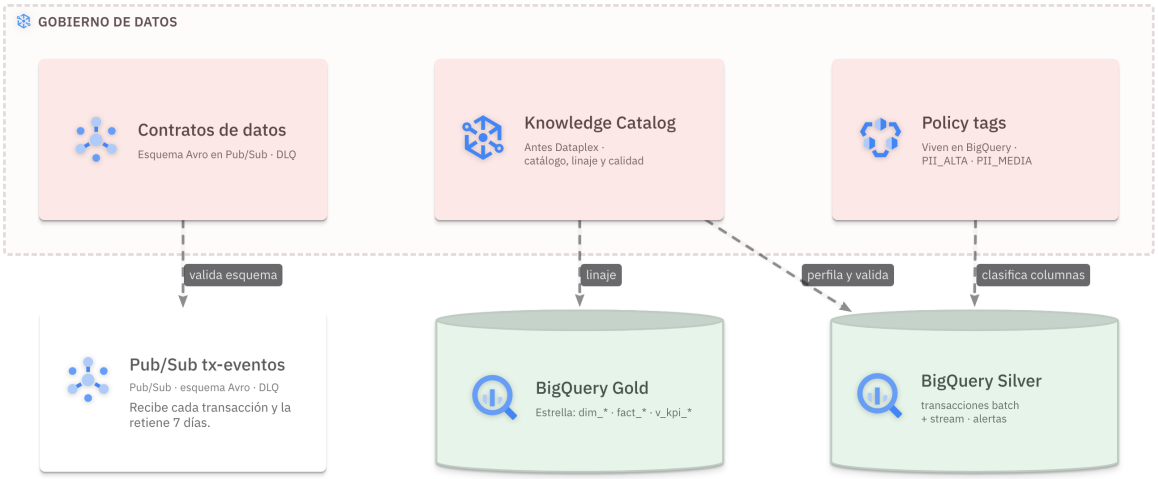


Figura 4. Capa de gobierno de datos

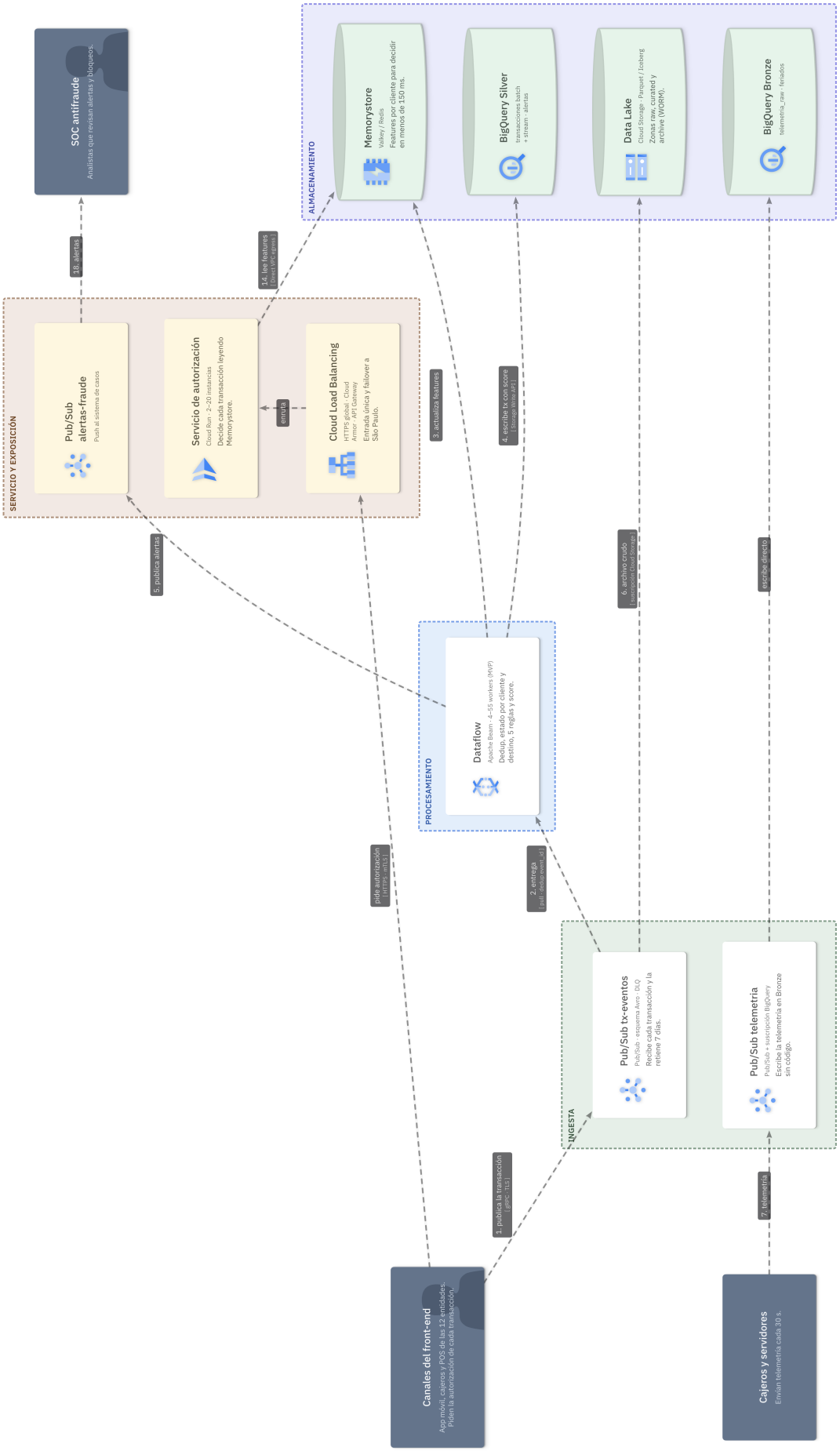


Figura 2. Contenedores del camino streaming (C4, nivel 2)

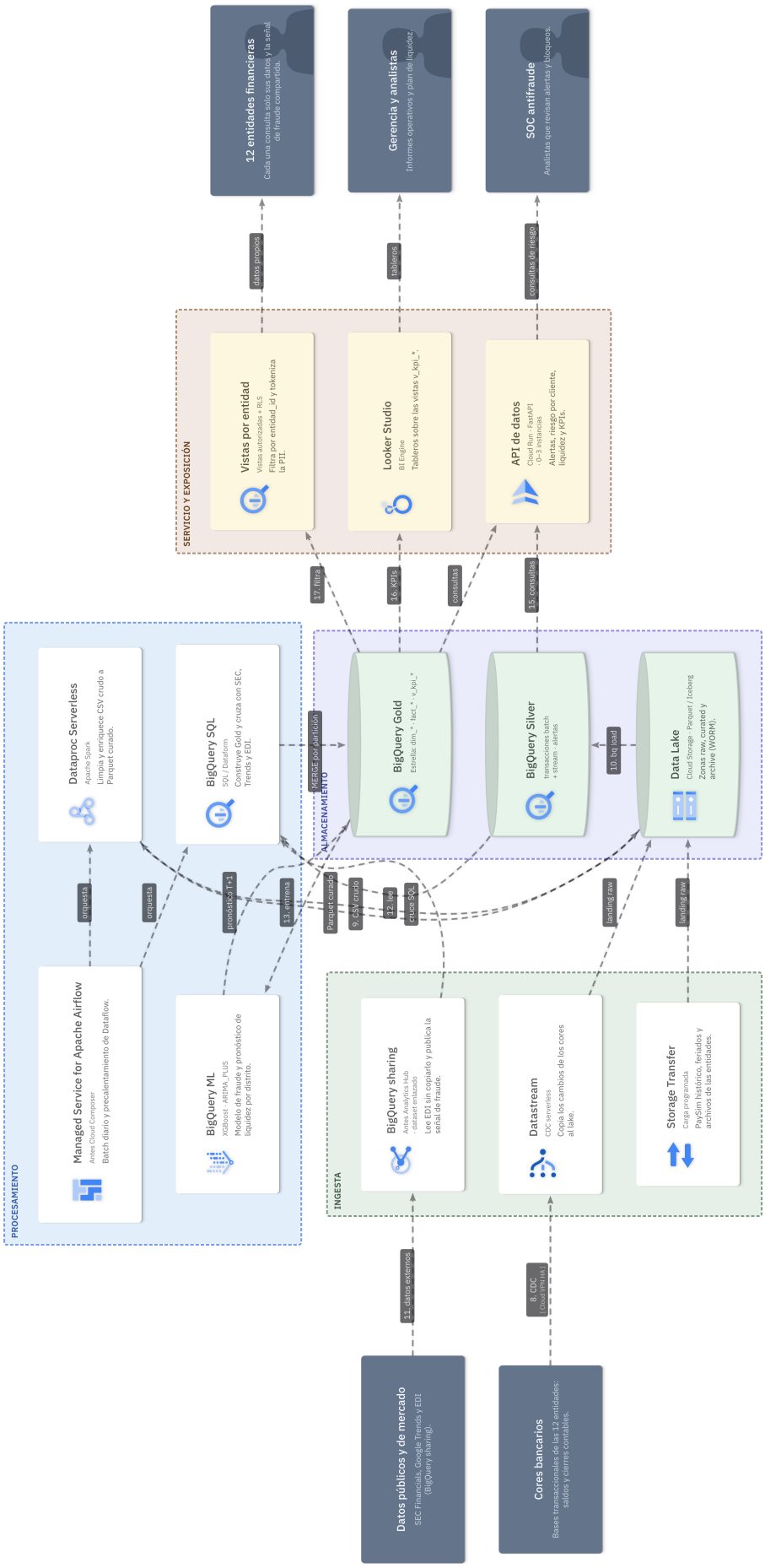


Figura 3. Contenedores del camino batch y de exposición (C4, nivel 2)

- **Knowledge Catalog** (antes Dataplex; el comando sigue siendo `gcloud dataplex`) cataloga las tablas, registra el linaje y corre el escaneo de calidad `lumina-dq-transacciones`: unicidad de `event_id`, nulos en `cliente_id`, montos no negativos, tipos válidos y distritos entre 1 y 8. Por presupuesto el escaneo evalúa una muestra del 10 % de `transacciones_batch`, así que no garantiza la unicidad global de `event_id`; esa la comprobamos con `COUNT(DISTINCT event_id)` sobre la tabla completa (figura 36).
- **Policy tags**: se crean en Knowledge Catalog (antes Data Catalog, que sigue siendo el nombre del servicio en IAM y en la API) pero se aplican sobre las columnas de BigQuery. La taxonomía `lumina_sensibilidad` distingue `PII_ALTA` (identificadores y saldos) de `PII_MEDIA` (ubicación y dispositivo). En el laboratorio creamos la taxonomía y los tags (figura 22), pero todavía no los asignamos a las columnas de BigQuery, así que el control de acceso por columna aún no opera.
- **Contratos de datos**: en producción un esquema de Pub/Sub en `tx-eventos` rechazará en la ingesta los mensajes con estructura o tipos inválidos. En el laboratorio ese esquema no está creado: toda la validación la hace la etapa `Parsear` de Dataflow (campos, tipos y montos positivos), que manda lo inválido al tópico `tx-eventos-dlq`. Con el esquema activo, a esa etapa solo le quedarían las reglas de dominio.
- **Señal de fraude compartida**: la vista `v_senal_fraude_compartida` publica destinos sospechosos sin datos personales. Responde a la queja del caso de que las entidades no comparten información de fraude.



Figura 5. Capa de seguridad

- **IAM**: el diseño pide una cuenta de servicio por componente, con roles sobre el dataset o el bucket que usa. En el laboratorio todos los pipelines comparten `sa-lumina-pipelines`, que tiene roles amplios a nivel de proyecto (`bigquery.dataEditor` y `storage.objectAdmin`, figura 21). Separarla es un pendiente.
- **Cloud KMS**: la llave `lumina-datos` rota cada 90 días y hoy cifra el dataset Gold y el bucket de archivo. Silver, Bronze y la zona raw siguen con llaves de Google; extender la llave a esas capas está en los pendientes.
- **Datos personales**: la vista `v_transacciones_entidad` filtra por la entidad del usuario, reemplaza el id del cliente por un seudónimo y redondea la ubicación.
- **Auditoría**: un sink manda los Cloud Audit Logs al dataset `lumina_auditoria`.

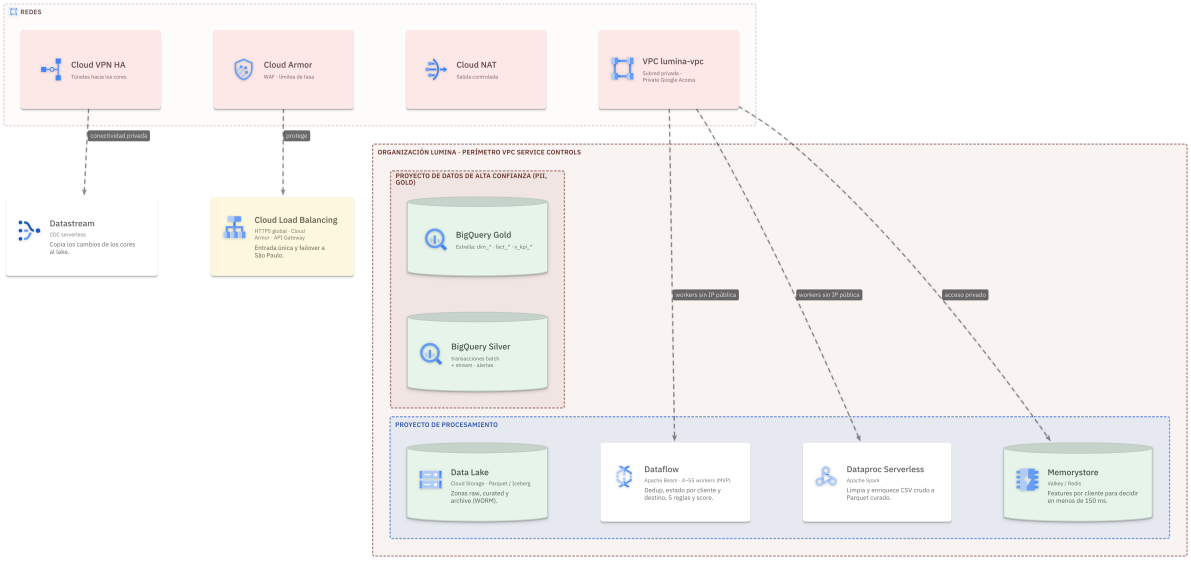


Figura 6. Capa de redes. El perímetro de VPC Service Controls se define en la organización y agrupa proyectos

- **VPC `lumina-vpc`** con subred privada y Private Google Access. Los workers de Dataflow corren sin IP pública dentro de ella desde la corrida de carga 2 (también la 3). *Desplegado.* Dataproc Serverless todavía no: el job batch se lanzó sin `--subnet` y corrió en la VPC `default`, con IP internas pero fuera de `lumina-vpc`.
- **Cloud VPN** lleva el tráfico de Datastream hacia los cores sin pasar por Internet. *Diseñado.*
- **VPC Service Controls** no se configura por servicio ni por proyecto sino en la organización: el perímetro agrupa proyectos y bloquea que los datos de BigQuery, Cloud Storage y Pub/Sub salgan de ellos, aunque la credencial sea válida. Planeamos dos perímetros, uno de alta confianza para el proyecto con PII y Gold y otro para el de procesamiento, con reglas de entrada y salida explícitas entre ellos (figura 6). *Diseñado.*
- **Cloud Load Balancing y Cloud Armor** son la única entrada pública y hacen el *failover* hacia São Paulo. *Diseñado.* Mientras no existan, la API de datos se publica en su URL de Cloud Run y solo la protege IAM: el tráfico no pasa por el WAF. En producción el servicio se configura con `--ingress internal-and-cloud-load-balancing`, de modo que la única entrada sea el balanceador.

Tabla 3. Qué quedó desplegado en el laboratorio y qué no

Componente	Estado	Motivo
Pub/Sub, Dataflow, Dataproc, BigQuery, BigQuery ML, Cloud Run, Cloud Storage	Desplegado	Núcleo de la entrega
VPC, firewall, KMS, auditoría, taxonomía de policy tags, Knowledge Catalog (calidad y linaje)	Desplegado	Capas transversales pedidas en la revisión
Asignación de policy tags a columnas	No	Se hace a mano en el esquema de cada tabla; queda para la Entrega 3
Dataproc en `lumina-vpc` y CMEK en el bucket curated	No	El batch corrió en la VPC `default` y curated usa llaves de Google; se corrige al pasar el ETL al DAG
Esquema de Pub/Sub en `tx-eventos`	No	La validación la hace Dataflow, con salida a la DLQ
Managed Service for Apache Airflow (antes Cloud Composer)	No	Costo: un entorno pequeño cuesta varias veces el presupuesto de S/ 180 del laboratorio. Los DAG están escritos y se mostrarán en la Entrega 3
Memorystore	No	Lo consume el servicio de autorización, que todavía no está desplegado; tiene costo fijo por hora
VPC Service Controls	No	Los perímetros se definen en una organización de Google Cloud; el proyecto está en una cuenta personal sin organización
Cloud VPN y Datastream	No	No hay cores bancarios reales a los que conectarse; el histórico entra como archivo
Load Balancing y Cloud Armor	No	Costo fijo mensual y una sola API de prueba; la API queda protegida solo por IAM

3 Por qué elegimos cada componente

Los criterios fueron cinco: la latencia que pide cada caso de uso, el volumen y su variación, el requisito de código abierto del caso, el trabajo de operación que le exige al equipo y el costo.

Tabla 4. Justificación técnica por componente

Componente	Qué hace en Lumina	Por qué este	Descartamos
Ingesta			
Pub/Sub	Recibe transacciones y telemetría y las retiene 7 días	Escala de 175 a 5 000 TPS sin dimensionar particiones. Tiene suscripciones nativas a BigQuery y Cloud Storage que nos ahorran dos pipelines	Kafka gestionado: hay que dimensionar brokers y particiones
Datastream	Copia los cambios de los cores	CDC sin instalar agentes en las bases de las entidades	Debezium: hay que operar Kafka Connect
Storage Transfer	Carga PaySim, feriados y archivos de las entidades	Programado, con reintentos y verificación	Scripts en una VM

Componente	Qué hace en Lumina	Por qué este	Descartamos
BigQuery sharing (antes Analytics Hub)	Lee EDI y publica la señal de fraude	Consume datos del proveedor sin copiarlos y comparte datos entre entidades con permisos	Copiar archivos
Procesamiento			
Dataflow (Apache Beam)	Evalúa el fraude en tiempo real	Estado y *timers* gestionados para viaje imposible y cuenta mula; escala horizontal. Beam también corre en Flink o Spark	Spark Streaming: micro-lotes y clúster permanente
Dataproc Serverless (Spark)	ETL batch de CSV a Parquet	Spark es el estándar y lo conocemos; se paga por job	Clúster fijo: se paga aunque no corra
BigQuery SQL	Construye Gold y cruza con datos públicos	Transforma donde está el dato, sin moverlo	Hacer todo en Spark
BigQuery ML	Fraude (XGBoost) y liquidez (ARIMA_PLUS)	Entrena con SQL sobre los mismos datos	Gemini Agent Platform (antes Vertex AI) desde el inicio: más trabajo para el MVP
Managed Service for Apache Airflow	Orquesta el batch y el precalentamiento	Airflow: dependencias, reintentos y *backfills*	Workflows: no hace *backfill*
Almacenamiento			
Memorystore	*Features* para autorizar en menos de 150 ms	Lecturas de submilisegundos; Valkey y Redis son abiertos	Bigtable: más latencia y mayor costo mínimo
Cloud Storage	Data Lake con zonas raw, curated y archive	Barato y durable, con ciclo de vida y bloqueo WORM; formatos abiertos	HDFS: une cómputo y almacenamiento
BigQuery	Bronze, Silver, Gold y modelos	Serverless, con seguridad por fila y columna y acceso directo a SEC y Trends	Hive o PostgreSQL: operación o escala
Exposición			
Cloud Run	Autorización y API de datos	Contenedores que escalan por concurrencia; la API escala a cero	GKE: un clúster que operar
Looker Studio	Tableros de gerencia	Sin licencia; con BI Engine responde en menos de un segundo	Looker: licencia
Vistas autorizadas	Datos por entidad	Control nativo sin copiar datos por banco	Un dataset por entidad
Capas transversales			
Knowledge Catalog	Catálogo, linaje y calidad	Integrado con BigQuery y Dataflow sin instalar nada	DataHub o Atlas: hay que desplegarlos
Cloud KMS	Llaves propias con rotación	Permite revocar el acceso y demostrar que la entidad controla las llaves de sus datos en la nube	Llaves gestionadas por Google
VPC Service Controls	Perímetro de datos a nivel de organización	Agrupa proyectos e impide copiar datos fuera aunque la credencial sea válida	Solo IAM

Componente	Qué hace en Lumina	Por qué este	Descartamos
Load Balancing + Armor	Entrada pública y *failover*	Balanceo global entre Santiago y São Paulo, con WAF	Balanceo regional

4 Dimensionamiento de Dataflow y Cloud Run

4.1 Volumen de referencia

Tabla 5. Escenarios de carga

Escenario	Cálculo	TPS
Promedio diario	15 M de transacciones ÷ 86 400 s	175
Hora pico de un día normal	2,5 × el promedio	440
Día de pago o cierre de mes	60 M × 12 % en la hora pico ÷ 3 600 s	2 000
Pico de diseño (Cyber Days)	2,5 × el día de pago	5 000

En la Entrega 1 sumamos la telemetría y el *clickstream* a la carga de Dataflow. Fue un error: la telemetría entra a BigQuery por una suscripción directa y el *clickstream* no pasa por el pipeline de fraude. Por Dataflow solo pasan transacciones.

4.2 Dataflow: lo que medimos

Al empezar esta entrega estimamos, sin medir, que un worker `n2-standard-2` procesaría unas 1 000 transacciones por segundo. Con eso bastaban 8 workers para el pico de diseño. Antes de fijar ese número lo pusimos a prueba.

Cómo medimos. Dejamos el job fijo en un solo worker (`--max_num_workers 1`) y publicamos transacciones de PaySim por escalones de tres minutos: 100, 300, 600 y 900 por segundo. En cada minuto registramos cuántos eventos llegaron a BigQuery, la latencia p95 entre el envío y la decisión, el backlog de la suscripción y el uso de CPU del worker. La consulta está en `src/sql/06_validacion_capacidad.sql`. Solo la corrida 1 recorrió el protocolo completo de 12 minutos. Como el worker se saturó ya en el primer escalón, las corridas 2 y 3 buscaban solo ver si el tope de 100 eventos por segundo cambiaba con cada ajuste, y las detuvimos antes: la 2 a los 6 minutos y la 3 a los 4. Por eso en la figura 7 se comparan los eventos por segundo sostenidos; la latencia p95 crece con el backlog acumulado y no es comparable entre corridas de distinta duración.

Qué encontramos. El worker se saturó en el primer escalón: le llegaban 100 eventos por segundo y procesaba entre 65 y 75. En la corrida 1 el simulador publicó 342 000 eventos en los cuatro escalones; el worker procesó 62 159 antes de que canceláramos el job y el backlog seguía creciendo (162 392 mensajes a las 04:37, figura 42). Hicimos tres corridas de carga (tabla 6 y figura 7). La 1 y la 3 cambian una sola cosa; la 2 cambió dos a la vez, los hilos y la red, así que su resultado no se puede atribuir a ninguna de las dos por separado.

Tabla 6. Corridas de la prueba de carga con un worker `n2-standard-2`

Corrida	Cambio	ev/s sostenidos	CPU	Qué aprendimos
1	Configuración por defecto	65–75 (pico de 143)	93–97 %	El worker no sigue el ritmo: el backlog llegó a 162 000 mensajes y la latencia p95, a casi 9 minutos. El runner advirtió que la caché de estado era insuficiente y sugirió `--max_cache_memory_usage_mb`.
2	64 hilos por proceso y, a la vez, workers sin IP pública en la VPC privada	55–63	94–96 %	No mejoró. La CPU al 95 % apunta a un límite de cómputo, pero no probamos la sugerencia del runner sobre la caché de estado, así que no podemos descartar que también influya. No medimos por separado el efecto de la red; es la configuración que usará producción, así que la tomamos como piso.
3	Escritura a BigQuery en lotes (`with_auto_sharding`), con la misma red de la 2 (sin IP pública en `lumina-vpc`)	78–89	—	Mejora cerca de un 15 %. El sink no era el problema principal.

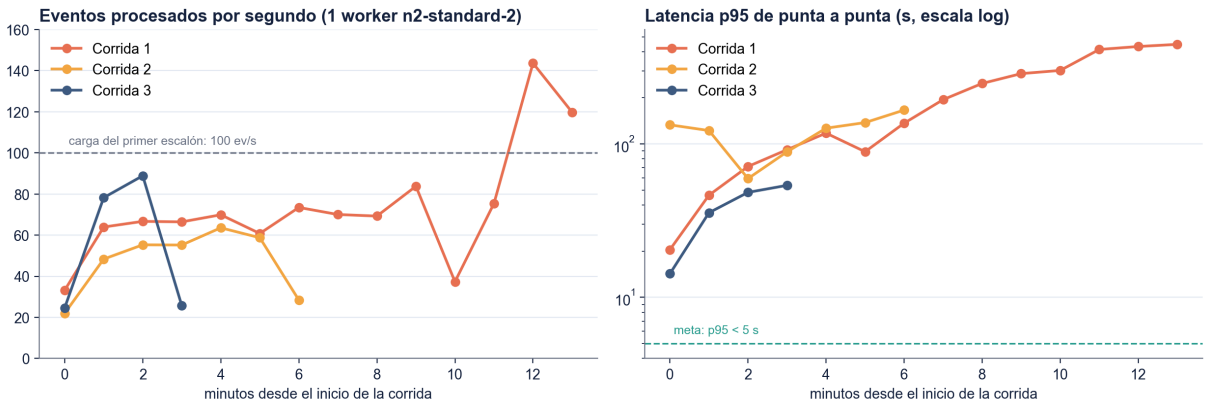


Figura 7. Eventos procesados y latencia p95 por minuto en las tres corridas

Con poca carga el pipeline sí cumple: a 5 eventos por segundo la latencia p95 fue de 2,4 s, por debajo de la meta de 5 s. El problema aparece al saturarse. Haciendo la cuenta (2 vCPU al 95 % para unos 70 eventos por segundo), cada transacción consume unos 27 ms de CPU. El costo está en las dos etapas con estado: por cada evento hay una lectura, una escritura y un *timer* por cliente y lo mismo por destino, y en el SDK de Python cada una de esas operaciones se serializa y viaja entre el proceso de Python y el *runner*. Escribimos ese código pensando en que fuera claro, no en que fuera rápido, y la prueba lo dejó a la vista.

Qué significa para el dimensionamiento. Con 75 eventos por segundo por worker, usados al 70 %, el código actual necesita 4 workers en promedio, 39 en un día de pago y 96 en el pico de diseño. Pero dentro de la VPC privada, que es como correrá en producción, el worker sostuvo entre 55 y 63. Para cubrir el día de pago con 40 workers haría falta que cada uno procese al menos 71,4 eventos

por segundo ($2\,000/(40 \times 0{,}7)$), y la corrida 2 no llega. Con el piso de 55, el día de pago pide 52 workers. Con la meta de optimización que fijamos más abajo, harían falta 1, 6 y 15 (figura 8).

Estas cifras suponen que la capacidad crece en línea recta con el número de workers. No lo demostramos: medimos con un solo worker. En Dataflow el estado se reparte por clave entre los workers y con 6,9 millones de clientes la carga queda bien distribuida, así que esperamos un comportamiento cercano al lineal, pero el *shuffle* y las escrituras a BigQuery pueden restar eficiencia. La prueba con autoscaling de la Entrega 3 lo confirmará.

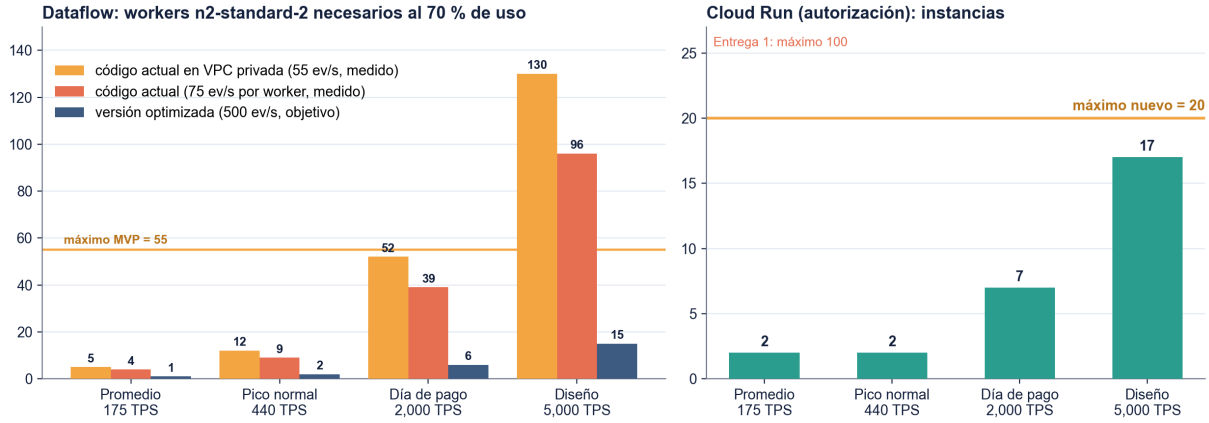


Figura 8. Workers de Dataflow e instancias de Cloud Run por escenario

Esto cambia la lectura de la Entrega 1. Allí el máximo de 30 workers `n2-standard-4` no salió de una medición: sobrestimaba la carga (contaba la telemetría) y suponía una capacidad por worker que el código no alcanza. Ahora los dos datos están medidos o corregidos, y la decisión es esta:

1. **Fase MVP, con el código actual**: mínimo de 4 workers, máximo de 55 y precalentamiento a 20 en días de pago con el DAG `lumina_precalentamiento_dataflow` (`src/orchestration/dag_lumina_precalentamiento.py`). El máximo sale del peor caso medido (52 workers con 55 eventos por segundo) más un margen pequeño, y cubre hasta el día de pago. Subirlo de 40 a 55 solo cuesta en las horas pico de esos días, unos USD 50 al mes, que caben en la contingencia del 15 % de la tabla 8. Si llegara un pico mayor, Pub/Sub retiene los mensajes sin perderlos, pero las alertas saldrían con minutos de retraso. Por eso esta versión no debe operar en un Cyber Day.
2. **Antes de producción**: optimizar el camino con estado hasta al menos 500 eventos por segundo por worker y repetir esta misma prueba. Las opciones, en el orden en que las probaremos: reescribir las dos etapas con estado en Beam para Java (el grafo y el diseño no cambian, y Beam corre igual en Dataflow); quitar el *timer* que se reprograma en cada evento y usar expiración de estado; y leer el historial del destino desde Memorystore en lugar de mantenerlo en el pipeline. Con 500 eventos por segundo, el máximo queda en 15 workers.

En el diseño, la autorización de cada transacción no depende de esto: la decide Cloud Run con las *features* de Memorystore en milisegundos, y Dataflow solo mantiene esas *features* al día y genera las alertas. Un atraso de Dataflow degradaría la calidad de la detección pero no detendría los pagos. Hoy esa protección no existe: ni el servicio de autorización ni Memorystore están desplegados, y el pipeline todavía no escribe en Memorystore (tabla 9). Hasta que lo estén, un atraso de Dataflow atrasa las alertas y no hay una segunda línea que decida en tiempo real.

4.3 Cloud Run

Usamos la ley de Little: solicitudes simultáneas = tasa de llegada × tiempo de respuesta. Las instancias quedan limitadas por la CPU o por la concurrencia, lo que llegue primero, y el autoscaler de Cloud Run apunta a un 60 % de uso.

Servicio de autorización. Hay una solicitud por transacción. Cada una tarda unos 25 ms (una lectura en Memorystore dentro de la VPC más las reglas) y consume unos 2 ms de CPU, así que una instancia de 1 vCPU atiende unas 500 solicitudes por segundo; al 60 % de uso quedan 300. En el pico de diseño hacen falta $\lceil 5\,000/300 \rceil = 17$ instancias y en un día de pago, 7. Fijamos un máximo de 20 (17 más un margen del 15 %) y un mínimo de 2, para no tener arranques en frío y seguir funcionando si cae una zona. La concurrencia baja de 80 a 40: con 25 ms por solicitud cada instancia tiene unas 8 en curso, y 80 no aportaba nada.

API de datos. La usan el SOC y las aplicaciones para consultas de riesgo; los tableros van directo a BigQuery. Estimamos 20 solicitudes por segundo en el pico y 1,5 s por consulta, o sea 30 solicitudes simultáneas. Como cada solicitud pasa casi todo el tiempo esperando a BigQuery, una instancia aguanta 20 en paralelo. Con 2 instancias basta; dejamos un máximo de 3, que además limita a 60 las consultas simultáneas contra BigQuery. El mínimo es 0 en el laboratorio y 1 en producción.

Tabla 7. Configuración de Cloud Run antes y después

Parámetro	Entrega 1	Autorización	API de datos
Instancias mínimas	2	2	0 (laboratorio), 1 (producción)
Instancias máximas	100	20	3
Concurrencia	80	40	20
CPU y memoria	—	1 vCPU, 512 MiB, CPU siempre asignada	1 vCPU, 512 MiB, CPU solo durante la solicitud

Bajar el máximo no reduce el costo por sí solo, porque Cloud Run cobra por uso. Lo que hace es poner un techo al gasto y proteger a Memorystore y BigQuery de una avalancha de solicitudes. La API ya está desplegada con esta configuración (figura 55).

4.4 Impacto en el costo

Recalculamos el costo mensual con precios de lista de `us-central1`, igual que en la Entrega 1, y lo llevamos a Santiago con un ajuste del 22 %. La tabla 8 muestra solo las partidas que cambian; el resto (Pub/Sub, Dataproc, BigQuery, Memorystore, Cloud Storage, Airflow gestionado, seguridad, observabilidad y DR) suma USD 2 635 al mes y no se modifica.

Tabla 8. Costo mensual estimado (USD)

Partida	Entrega 1	MVP (código actual)	Producción (optimizado)
Dataflow	1 100	800	250
Cloud Run, balanceador y Cloud Armor	80	160	160
Redes (VPN hacia los cores, NAT, egress)	80	180	180
Resto de servicios	2 635	2 635	2 635
Subtotal más 15 % de contingencia	4 480	4 340	3 710
Santiago con DR en São Paulo	**5 500**	**5 300**	**4 500**
Año 1	**66 000**	**63 600**	**54 000**
USD por millón de transacciones	12	12	10

Dataflow en el MVP supone unos 4,5 workers en promedio a lo largo del día, más los picos de los días de pago. Cloud Run sube respecto de la Entrega 1 porque ahora contamos bien las dos instancias mínimas del servicio de autorización, y la partida de redes crece porque incluye la VPN hacia los cores. Son estimaciones que hay que confirmar en la calculadora de precios de Google Cloud.

El laboratorio corre bajo un presupuesto de S/ 180 con alertas al 50, 90 y 100 % (figura 9). Dataflow se apagó al terminar cada prueba; la API escala a cero cuando nadie la usa.

5 Implementación de la Entrega 2

5.1 Pipeline batch con Spark

El script `src/dataproc/batch_paysim_etl.py` lee el CSV de PaySim con un esquema explícito y escribe Parquet particionado por fecha. Descarta filas sin llaves o monto, tipos fuera de dominio, montos negativos y duplicados. Genera un `event_id` determinista, convierte el paso de simulación en fecha real y asigna distrito, entidad, edad, canal y ubicación con el mismo hash MD5 que usan el simulador y el SQL. También calcula los descuadres de saldo, que son la señal más fuerte de fraude en PaySim.

El job de Dataproc Serverless tardó 2 min 51 s, leyó 6 362 620 filas y las escribió todas en 31 particiones (552 MiB en Parquet). Ninguna fila cayó en las reglas de descarte del ETL, pero eso no significa que el archivo viniera limpio: tuvimos que normalizarlo a mano antes de subirlo, fuera del pipeline, como se explica a continuación. Después, `bq load` llevó el Parquet a `lumina_silver.transacciones_batch`, particionada por día y agrupada por distrito, tipo y cliente.

Al descargar el dataset de Kaggle encontramos dos problemas que obligaron a normalizarlo antes de subirlo: viene partido en `train.csv` y `validation.csv`, con las columnas `isFraud` e `isFlaggedFraud` en orden invertido, y con fines de línea de Windows. Como el ETL lee por posición, sin esa limpieza la etiqueta de fraude habría quedado cambiada. Esa normalización la hicimos con un script aparte; en la Entrega 3 la pasaremos al ETL, que leerá las columnas por nombre.

5.2 Pipeline streaming con Apache Beam

Tabla 9. Pasos del pipeline `fraud_streaming_pipeline.py`

Paso	Qué hace
`LeerPubSub`	Lee la suscripción y descarta duplicados por `event_id`
`Parsear`	Valida campos, tipos y montos; lo inválido va al tópico `tx-eventos-dlq`
`ViajeImposible`	Guarda por cliente la última ubicación y calcula la velocidad
`VelocidadDestino`	Cuenta por destino las transacciones de los últimos 10 minutos
`Reglas`	Suma los pesos de las reglas de la tabla 10 en un score de 0 a 1: desde 0,8 bloquea, desde 0,5 manda a revisión
Salidas	Todas las transacciones a Silver; las alertas a Silver y al tópico `alertas-fraude`. La escritura de *features* a Memorystore está en el diseño, no en el código actual

En la prueba funcional el simulador publicó 30 000 transacciones a 100 por segundo y las 30 000 llegaron a Silver con `event_id` distintos; 130 terminaron en alerta (50 bloqueos y 80 a revisión, figura 36). La figura 37 muestra 29 984 y 122 porque filtra por la hora del evento y deja fuera los últimos segundos de la corrida. En paralelo, 50 terminales enviaron 6 000 lecturas de telemetría que la suscripción escribió en Bronze sin pasar por ningún pipeline.

Tabla 10. Reglas de fraude del pipeline y su peso en el score

Regla	Se activa cuando	Peso
`R1_VACIADO_CUENTA`	TRANSFER o CASH_OUT por todo el saldo de origen (la cuenta queda en cero)	0,5
`R2_INCONSISTENCIA_DESTINO`	TRANSFER o CASH_OUT hacia un destino con saldo cero antes y después	0,3
`R3_MONTO_ALTO`	TRANSFER de más de 200 000	0,2
`R4_VIAJE_IMPOSIBLE`	Entre dos transacciones del mismo cliente la velocidad supera 800 km/h	0,5
`R5_CUENTA_MULA`	Tres o más TRANSFER al mismo destino en 10 minutos	0,3

5.3 Batch y streaming en el mismo modelo

Los dos caminos escriben en Silver con el mismo esquema y asignan distrito y entidad con la misma función. La vista `fact_transacciones` los une con una columna `origen`, de modo que un KPI calculado sobre el histórico se puede comparar con uno calculado en vivo. Revisamos que la unión no duplique transacciones: ningún `event_id` aparece en los dos orígenes, y el histórico cubre julio de 2026 mientras que el stream es del 10 de noviembre, así que tampoco se cruzan en los KPI diarios (figura 36).

5.4 Modelo estrella en Gold

Tabla 11. Tablas y vistas de la capa Gold

Objeto	Grano	Uso
`dim_distrito`	8 distritos	Liquidez y congestión; cada uno se asocia a una región de Google Trends
`dim_entidad`	12 entidades	Acceso por entidad; cada una tiene un banco de la SEC como referencia
`dim_cliente`	6,9 M de clientes	Segmentación por edad, distrito y entidad
`dim_fecha`	Día	Feriados y días de pago
`fact_transacciones`	Transacción	Une batch y streaming
`fact_liquidez_hora`	Distrito × hora	Entrada del pronóstico de efectivo
`fact_tendencias_semana`	Distrito × semana	Demanda digital según Google Trends
`fact_benchmark_sector`	Banco × año	ROA y apalancamiento del sector
`v_kpi_*` y vistas de gobierno	—	Informes operativos, reglas de fraude, acceso por entidad

Dos ajustes respecto a lo planeado. Google Trends no publica datos de Perú en `international_top_terms`, así que usamos Colombia como referencia andina (33 regiones). Y para elegir los bancos de referencia usamos los informes 10-K de 2018, el último año con cobertura completa en el dataset de la SEC; con 2020 solo aparecían 4 bancos.

5.5 Modelos en BigQuery ML

El modelo de fraude (XGBoost) se evaluó sobre el 20 % de TRANSFER y CASH_OUT que no vio en el entrenamiento, con la proporción real de fraude. Obtuvo una precisión de 0,81, un recall de 0,995 y un AUC de 0,998, por encima de la meta de la Entrega 1 (recall de al menos 85 % con precisión de al menos 70 %). Ese resultado tiene una fuga de información. Las dos variables que más pesan son el descuadre del saldo de origen (`err_saldo_orig`, 2,31) y el saldo de origen después de la transacción (`saldo_nuevo_orig`, 0,44), y ninguna existe en el momento de autorizar: se conocen cuando la transacción ya se ejecutó. Entrenamos un segundo modelo solo con lo que se sabe antes de decidir (tipo, monto, saldos previos, proporción del saldo que se mueve y hora). Obtuvo una precisión de 0,81, un recall de 0,994 y un AUC de 0,998 (figura 52), prácticamente lo mismo. La variable que manda ahora es la proporción del saldo que se mueve: en PaySim el fraude casi siempre vacía la cuenta, el mismo patrón que usa la regla R1. Con datos reales no esperamos una separación tan limpia; la cifra sirve para validar el flujo, no para prometer ese desempeño.

Las reglas del pipeline, sin modelo, detectan el 98,9 % de los fraudes con una precisión de 0,52: por cada fraude detectado generan 0,9 falsas alarmas. El modelo genera 0,23, cerca de un 74 % menos. La comparación es aproximada porque las bases no son iguales. Las reglas se midieron sobre las 92 074 transacciones reales de PaySim que habían pasado por el pipeline cuando tomamos la captura (figura 48): las 30 000 de la prueba funcional, publicadas con `--solo-transfer-cashout`, y las 62 159 de la corrida de carga 1, menos 85 eventos sintéticos. La vista excluye los sintéticos, que son los PAYMENT que el simulador inyecta antes de un viaje imposible. El modelo se midió sobre el 20 % reservado de TRANSFER y CASH_OUT del histórico.

Los conteos del camino streaming cambian entre figuras porque `transacciones_stream` creció mientras capturábamos: 92 074 filas evaluables en la figura 48 (23:38 del 9 de noviembre), 97 708 en la figura 47 (23:44, con la corrida 2 en curso) y 125 274 en la figura 36, que tomamos el 10 de

noviembre con las tres corridas de carga ya cargadas. Las vistas de Gold agregan sobre la misma tabla; la diferencia es solo el momento del corte.

El pronóstico de liquidez (ARIMA_PLUS por distrito) no alcanza la meta de un error menor al 15 %. Con los cuatro días de evaluación, el MAPE diario va de 70 % a 816 % según el distrito. Si se excluye el 31 de julio baja a entre 32 % y 59 % en siete distritos y queda en 228 % en el octavo (figura 53). Ninguna de las dos lecturas cumple la meta. La causa está en los datos: en sus últimos días PaySim casi no tiene movimiento (282 transacciones el 31 de julio frente a unas 50 000 en un día típico), y esos son justamente los días reservados para evaluar. Con un año de datos reales el modelo podrá aprender la estacionalidad semanal y mensual; con 27 días no puede.

La vista `v_plan_liquidez_t1` tenía un error que muestra la figura 51: el efectivo recomendado salía entre 10 y 36 veces el esperado y crecía cada día. Calculábamos el recomendado sumando las 24 cotas superiores horarias al 90 %, y la suma de esos cuantiles no es el cuantil del total del día. Además la vista devolvía los 7 días del horizonte, y el intervalo de ARIMA se ensancha con cada hora que se aleja. La corregimos: ahora devuelve solo el primer día pronosticado (T+1) y suma al esperado un margen igual a la raíz de la suma de los cuadrados de los márgenes horarios, suponiendo errores horarios independientes. La captura de la figura 51 es anterior a la corrección.

5.6 API de datos

La API en Cloud Run responde 403 a quien llama sin credenciales y devuelve las alertas, el riesgo de un cliente y el plan de liquidez a quien presenta un token válido (figura 58). Hoy se accede directo por su URL de Cloud Run, sin pasar por Cloud Armor (sección 2.5). Tiene además dos problemas de gobierno que corregiremos: corre con la cuenta compartida `sa-lumina-pipelines`, que tiene permisos de escritura en todo el proyecto, y consulta Silver directamente, así que devuelve `cliente_id` en claro (figura 58) en lugar de pasar por las vistas seudonimizadas. Debe usar una cuenta propia con lectura solo sobre esas vistas. En la Entrega 3 la conectaremos al tablero y a la demo.

6 Evidencias de la implementación

Todas las capturas son salidas reales de `gcloud` y `bq` ejecutadas sobre el proyecto `lumina-bank-utec` el 9 y 10 de noviembre de 2026. Cada una muestra el comando, la fecha y el resultado. Los logs completos están en `doc/evidencias/logs/`. Las figuras marcadas como “Consola” son capturas de la consola web de Google Cloud y complementan la salida de terminal del mismo recurso.

6.1 Proyecto e infraestructura base

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud billing budgets create --billing-account=01D01F-87B06E-355892 --display-name="Lumina Bank UTEC" --budget-amount=180PEN
--filter-projects=projects/lumina-bank-utec --threshold-rule=percent=0.5 --threshold-rule=percent=0.9 --threshold-
rule=percent=1.0 --format="yaml(displayName,amount,thresholdRules,budgetFilter.projects)"
# proyecto: lumina-bank-utec · 2026-11-09 22:48:57 -05

Created [f9728553-1299-4979-9142-903bcb30b75e].
amount:
  specifiedAmount:
    currencyCode: PEN
    units: '180'
budgetFilter:
  projects:
  - projects/678938254786
displayName: Lumina Bank UTEC
thresholdRules:
- spendBasis: CURRENT_SPEND
  thresholdPercent: 0.5
- spendBasis: CURRENT_SPEND
  thresholdPercent: 0.9
- spendBasis: CURRENT_SPEND
  thresholdPercent: 1.0

# código de salida: 0
```

Figura 9. Presupuesto de S/ 180 con alertas al 50, 90 y 100 %. La cuenta de facturación está en soles

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud services list --enabled --format="table(config.name,config.title)" | grep -E "NAME|pubsub|dataflow|dataproc|bigquery.go
ogleapis|run.googleapis|cloudkms|dlp|dataplex|storage.googleapis|analyticshub|artifactregistry|cloudbuild"
# proyecto: lumina-bank-utec · 2026-11-09 22:53:29 -05

NAME                                TITLE
analyticshub.googleapis.com         Analytics Hub API
artifactregistry.googleapis.com     Artifact Registry API
bigquery.googleapis.com             BigQuery API
bigquerystorage.googleapis.com      BigQuery Storage API
cloudbuild.googleapis.com           Cloud Build API
cloudkms.googleapis.com             Cloud Key Management Service (KMS) API
dataflow.googleapis.com             Dataflow API
dataplex.googleapis.com             Cloud Dataplex API
dataproc-control.googleapis.com     Cloud Dataproc Control API
dataproc.googleapis.com             Cloud Dataproc API
dataprocrm.googleapis.com           Dataproc Resource Manager API
dlp.googleapis.com                  Sensitive Data Protection (DLP)
pubsub.googleapis.com               Cloud Pub/Sub API
run.googleapis.com                  Cloud Run Admin API
storage.googleapis.com              Cloud Storage API

# código de salida: 0
```

Figura 10. APIs habilitadas en el proyecto

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bash infra/setup.sh
# proyecto: lumina-bank-utec · 2026-11-09 22:49:07 -05

Updated property [core/project].
==> Habilitando APIs
Operation "operations/acf.p2-678938254786-2d1ab57c-d27c-4f05-bf8f-faa2bc4f8be0" finished successfully.
==> Buckets del Data Lake (multirregión US, acceso uniforme)

Updating gs://lumina-bank-utec-lumina-raw/...

Updating gs://lumina-bank-utec-lumina-archive/...
==> Cuenta de servicio de pipelines (mínimo privilegio)
Updated IAM policy for project [lumina-bank-utec].
Updated IAM policy for project [lumina-bank-utec].
Updated IAM policy for project [lumina-bank-utec].
Updated IAM policy for project [lumina-bank-utec].
Updated IAM policy for project [lumina-bank-utec].
Updated IAM policy for project [lumina-bank-utec].
Updated IAM policy for project [lumina-bank-utec].
==> Permisos del agente de Pub/Sub (suscripciones a BigQuery y Cloud Storage)
Updated IAM policy for project [lumina-bank-utec].
==> BigQuery: datasets, Bronze y Silver



[… 80 líneas más en el log …]
```

Figura 11. Ejecución de `infra/setup.sh`: APIs, buckets, cuenta de servicio y datasets

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud storage buckets list --format="table(name,location,storageClass,versioning_enabled,uniform_bucket_level_access)"
# proyecto: lumina-bank-utec · 2026-11-09 22:53:12 -05

NAME                           LOCATION  STORAGE_CLASS  VERSIONING_ENABLED  UNIFORM_BUCKET_LEVEL_ACCESS
lumina-bank-utec-lumina-archive  US                                       True
lumina-bank-utec-lumina-curated  US                                       True
lumina-bank-utec-lumina-raw      US                     True              True
lumina-bank-utec-lumina-temp     US                                       True

# código de salida: 0
```

Figura 12. Los cuatro buckets del Data Lake en la multirregión US; `raw` tiene versionado

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud storage buckets describe gs://lumina-bank-utec-lumina-archive --format="yaml(lifecycle_config)"
# proyecto: lumina-bank-utec · 2026-11-09 22:53:13 -05

lifecycle_config:
  rule:
  - action:
      storageClass: NEARLINE
      type: SetStorageClass
    condition:
      age: 30
  - action:
      storageClass: COLDLINE
      type: SetStorageClass
    condition:
      age: 90
  - action:
      storageClass: ARCHIVE
      type: SetStorageClass
    condition:
      age: 365

# código de salida: 0
```

Figura 13. Ciclo de vida del bucket de archivo: Nearline a 30 días, Coldline a 90 y Archive a 365

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud pubsub topics list --format="table(name.basename(),messageRetentionDuration)"; echo; gcloud pubsub subscriptions list
--format="table(name.basename(),topic.basename(),bigqueryConfig.table,cloudStorageConfig.bucket,ackDeadlineSeconds)"
# proyecto: lumina-bank-utec · 2026-11-09 22:53:15 -05

NAME           MESSAGE_RETENTION_DURATION
tx-eventos     604800s
tx-eventos-dlq  604800s
alertas-fraude  604800s
telemetria     604800s

NAME                    TOPIC           TABLE                                    BUCKET
ACK_DEADLINE_SECONDS
tx-eventos-archivo      tx-eventos                                               lumina-bank-utec-lumina-raw  300
tx-eventos-dlq-revision  tx-eventos-dlq                                                                       10
tx-eventos-dataflow     tx-eventos                                                                            60
alertas-fraude-demo     alertas-fraude                                                                        10
telemetria-bq           telemetria      lumina-bank-utec:lumina_bronze.telemetria_raw                         10

# código de salida: 0
```

Figura 14. Tópicos y suscripciones de Pub/Sub: Dataflow, archivo en Cloud Storage y escritura directa a BigQuery

6.2 Gobierno, seguridad y redes

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bash infra/gobierno_seguridad_red.sh
# proyecto: lumina-bank-utec · 2026-11-10 00:20:03 -05

Operation "operations/acat.p2-678938254786-66d35687-239c-49e4-8123-a9229a6f6e3e" finished successfully.
==> VPC privada y subred con Private Google Access
   lumina-vpc ya existe
   lumina-datos-us-central1 ya existe
==> Cloud KMS: llave CMEK con rotación cada 90 días
   keyring ya existe
   llave ya existe

Updating gs://lumina-bank-utec-lumina-archive/...
Dataset 'lumina-bank-utec:lumina_gold' successfully updated.
==> Auditoría: logs de Cloud Audit → BigQuery (lumina_auditoria)
   lumina_auditoria ya existe
==> Policy tags (Data Catalog): taxonomía de sensibilidad
   la taxonomía ya existe (o falló la creación): revise Dataplex → Policy tags
==> Dataplex: escaneo de calidad sobre lumina_silver.transacciones_batch
   el escaneo ya existe
projects/678938254786/locations/us-central1/dataScans/lumina-dq-transacciones/jobs/e0ed8781-3b71-4096-ad50-ddd9d610be76
==> Vistas por entidad y señal de fraude compartida



CREATE TABLE IF NOT EXISTS `lumina-bank-utec.lumina_gold.acceso_entidad` (
  usuario STRING NOT NULL OPTIONS (description = 'correo del usuario (SESSION_USER)'),
  entidad_id INT64 NOT NULL
) OPTIONS (description = 'Gobierno: mapeo usuario → entidad para la vista por entidad'); -- at [7:1]
Skipped lumina-bank-utec.lumina_gold.acceso_entidad

CREATE OR REPLACE VIEW `lumina-bank-utec.lumina_gold.v_transacciones_entidad`
OPTIONS (description = 'Seguridad: filas filtradas por entidad del usuario y PII tokenizada') AS
SELECT t.event_id, t.event_ts, t.fecha, t.tipo, t.monto, t.canal, t.entidad_id, t.distrito_id,
       TO_HEX(SHA256(CONCAT('lumina-salt', t.cliente_id))) AS cliente_token,  -- seudónimo estable
       ROUND(t.lat, 2) AS lat_aprox, ROUND(t.lon, 2) AS lon_aprox,           -- geo con menor precisión
       t.score, t.decision, t.origen
FROM `lumina-bank-utec.lumina_gold.fact_transacciones` t

[… 25 líneas más en el log …]
```

Figura 15. Segunda ejecución de `infra/gobierno_seguridad_red.sh`: el script detecta lo que ya creó y no lo duplica

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud compute networks subnets describe lumina-datos-us-central1 --region us-central1
--format='yaml(name,network.basename(),ipCidrRange,privateIpGoogleAccess,logConfig.enable)'; echo; gcloud compute firewall-rules
 list --filter='network:lumina-vpc' --format='table(name,direction,sourceRanges.list(),allowed[].map().firewall_rule().list())'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:24 -05

ipCidrRange: 10.10.0.0/20
logConfig:
  enable: true
name: lumina-datos-us-central1
network: lumina-vpc
privateIpGoogleAccess: true

NAME            DIRECTION  SOURCE_RANGES    ALLOW
lumina-iap-ssh  INGRESS    35.235.240.0/20  tcp:22
lumina-interno  INGRESS    10.10.0.0/20     tcp,udp,icmp

# código de salida: 0
```

Figura 16. Subred privada con Private Google Access y reglas de firewall de `lumina-vpc`

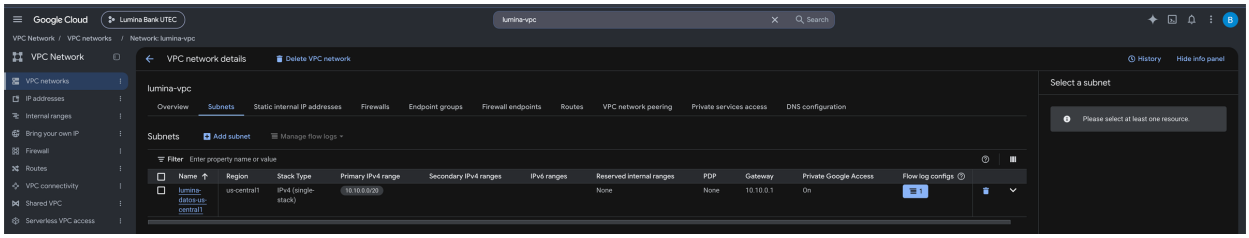


Figura 17. Consola: subred `lumina-datos-us-central1` (10.10.0.0/20) con Private Google Access activo y flow logs

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud kms keys describe lumina-datos --keyring lumina-kr --location us
--format='yaml(name,purpose,rotationPeriod,nextRotationTime,primary.state,primary.algorithm)'; echo; gcloud kms keys get-iam-
policy lumina-datos --keyring lumina-kr --location us --format='yaml(bindings)'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:27 -05

name: projects/lumina-bank-utec/locations/us/keyRings/lumina-kr/cryptoKeys/lumina-datos
nextRotationTime: '2027-02-08T04:25:48Z'
primary:
  algorithm: GOOGLE_SYMMETRIC_ENCRYPTION
  state: ENABLED
purpose: ENCRYPT_DECRYPT
rotationPeriod: 7776000s

bindings:
- members:
  - serviceAccount:bq-678938254786@bigquery-encryption.iam.gserviceaccount.com
  - serviceAccount:service-678938254786@gs-project-accounts.iam.gserviceaccount.com
  role: roles/cloudkms.cryptoKeyEncrypterDecrypter

# código de salida: 0
```

Figura 18. Llave `lumina-datos`: rotación cada 90 días y permisos solo para los agentes de BigQuery y Cloud Storage

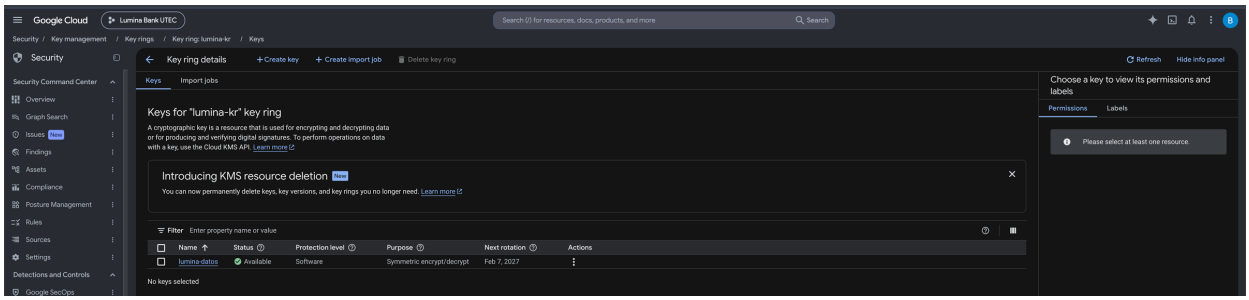


Figura 19. Consola: llave `lumina-datos` disponible en el keyring `lumina-kr`, con la próxima rotación programada para el 7 de febrero de 2027 a las 23:25 en hora de Lima (UTC-5); la CLI de la figura 18 muestra la misma fecha en UTC, 8 de febrero a las 04:25

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bq show --format=prettyjson lumina-bank-utec:lumina_gold | python3 -c 'import sys,json;d=json.load(sys.stdin);print("dataset:"
,d["id"]);print("defaultEncryptionConfiguration:",d.get("defaultEncryptionConfiguration"))'; gcloud storage buckets describe
gs://lumina-bank-utec-lumina-archive --format='yaml(name,default_kms_key)'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:30 -05

dataset: lumina-bank-utec:lumina_gold
defaultEncryptionConfiguration: {'kmsKeyName': 'projects/lumina-bank-utec/locations/us/keyRings/lumina-kr/cryptoKeys/lumina-
datos'}
default_kms_key: projects/lumina-bank-utec/locations/us/keyRings/lumina-kr/cryptoKeys/lumina-datos
name: lumina-bank-utec-lumina-archive

# código de salida: 0
```

Figura 20. El dataset Gold y el bucket de archivo usan la llave propia (CMEK)

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud projects get-iam-policy lumina-bank-utec --flatten='bindings[].members' --filter='bindings.members:sa-lumina-pipelines'
 --format='table(bindings.role)'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:33 -05

ROLE
roles/bigquery.dataEditor
roles/bigquery.jobUser
roles/dataflow.worker
roles/dataproc.worker
roles/pubsub.publisher
roles/pubsub.subscriber
roles/pubsub.viewer
roles/storage.objectAdmin

# código de salida: 0
```

Figura 21. Roles de la cuenta de servicio de los pipelines

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ TAX=$(curl -s -H "Authorization: Bearer $(gcloud auth print-access-token)"
https://datacatalog.googleapis.com/v1/projects/lumina-bank-utec/locations/us/taxonomies | python3 -c 'import
sys,json;t=json.load(sys.stdin)["taxonomies"][0];print(t["name"])'); curl -s -H "Authorization: Bearer $(gcloud auth print-
access-token)" https://datacatalog.googleapis.com/v1/$TAX | python3 -c 'import
sys,json;t=json.load(sys.stdin);print("taxonomía:",t["displayName"],"·",t.get("activatedPolicyTypes"))'; curl -s -H
"Authorization: Bearer $(gcloud auth print-access-token)" https://datacatalog.googleapis.com/v1/$TAX/policyTags | python3 -c
'import sys,json;[print(" -",p["displayName"],":",p.get("description","")) for p in json.load(sys.stdin)["policyTags"]]'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:35 -05

taxonomía: lumina_sensibilidad · ['FINE_GRAINED_ACCESS_CONTROL']
 - PII_ALTA : montos y saldos por cliente, identificadores
 - PII_MEDIA : geolocalización y dispositivo

# código de salida: 0
```

Figura 22. Taxonomía de sensibilidad con los tags `PII_ALTA` y `PII_MEDIA`

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud dataplex datascans jobs describe 809ee69a-eae5-4359-9786-e5940b1f5148 --datascan=lumina-dq-transacciones --location=us-
central1 --view=FULL --format=json | python3 ../doc/evidencias/dq_resultado.py
# proyecto: lumina-bank-utec · 2026-11-10 00:08:56 -05

job: 809ee69a-eae5-4359-9786-e5940b1f5148 · estado: SUCCEEDED · filas evaluadas: 636421
resultado global: APROBADO · puntaje: 100.0

columna       dimensión     regla                    evaluadas  aprobadas  resultado
event_id      UNIQUENESS    uniquenessExpectation       636421     636421  OK
cliente_id    COMPLETENESS  nonNullExpectation          636421     636421  OK
monto         VALIDITY      rangeExpectation            636421     636421  OK
tipo          VALIDITY      setExpectation              636421     636421  OK
distrito_id   VALIDITY      rangeExpectation            636421     636421  OK

# código de salida: 0
```

Figura 23. Escaneo de calidad de Knowledge Catalog (`gcloud dataplex`) sobre Silver: cinco reglas aprobadas en una muestra de 636 421 filas

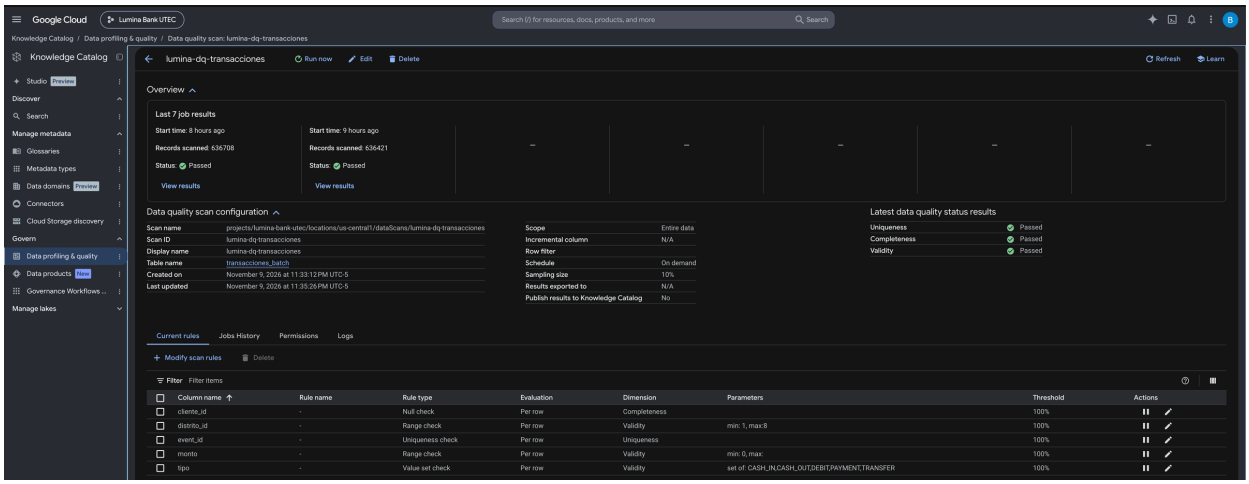


Figura 24. Consola de Knowledge Catalog: dos ejecuciones del escaneo aprobadas y las cinco reglas con umbral de 100 %

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bq query --use_legacy_sql=false --format=pretty 'SELECT entidad_id, origen, COUNT(*) AS filas FROM
lumina_gold.v_transacciones_entidad GROUP BY 1,2'; bq query --use_legacy_sql=false --format=pretty 'SELECT event_id, tipo,
monto, entidad_id, SUBSTR(cliente_token,1,16) AS cliente_token, lat_aprox, lon_aprox FROM lumina_gold.v_transacciones_entidad
LIMIT 5'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:40 -05

+------------+--------+--------+
| entidad_id | origen | filas  |
+------------+--------+--------+
|          3 | stream |   7672 |
|          3 | batch  | 529438 |
+------------+--------+--------+
+----------------------------------+---------+----------+------------+------------------+-----------+-----------+
|             event_id             |  tipo   |  monto   | entidad_id |  cliente_token   | lat_aprox | lon_aprox |
+----------------------------------+---------+----------+------------+------------------+-----------+-----------+
| 372b87e127e8f93c55584e9bb46a9f15 | PAYMENT | 10904.31 |          3 | b4be4d098f16b406 |    -11.93 |    -76.98 |
| e885ae5dc5563975733f0c2014cc2fb1 | PAYMENT |   387.95 |          3 | 088062e680983c95 |    -11.93 |     -77.0 |
| ee065863d3419dcc50dae5540159c1ad | PAYMENT |  9235.84 |          3 | b59fdc8436d64eb8 |    -11.94 |    -76.98 |
| 9e4ac8debdfb62a73e76b48d6b8df8a3 | PAYMENT | 17513.96 |          3 | 418802319f304a34 |    -11.93 |     -77.0 |
| e8e0238b6d99ba98c8ca2ef6dbefb7b0 | PAYMENT |    23505 |          3 | 861e3d3e41963b3b |    -11.95 |     -77.0 |
+----------------------------------+---------+----------+------------+------------------+-----------+-----------+

# código de salida: 0
```

Figura 25. La vista por entidad solo devuelve filas de la entidad 3 y reemplaza el id del cliente por un seudónimo

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud logging sinks describe lumina-auditoria --format='yaml(name,destination,filter,writerIdentity,bigqueryOptions)'; bq ls
lumina_auditoria
# proyecto: lumina-bank-utec · 2026-11-09 23:39:45 -05

bigqueryOptions:
  usePartitionedTables: true
  usesTimestampColumnPartitioning: true
destination: bigquery.googleapis.com/projects/lumina-bank-utec/datasets/lumina_auditoria
filter: logName:"cloudaudit.googleapis.com"
name: lumina-auditoria
writerIdentity: serviceAccount:service-678938254786@gcp-sa-logging.iam.gserviceaccount.com
                tableId                 Type    Labels     Time Partitioning      Clustered Fields
 ---------------------------------------- ------- -------- ---------------------- -----------------
  cloudaudit_googleapis_com_activity      TABLE            DAY (field: timestamp)
  cloudaudit_googleapis_com_data_access   TABLE            DAY (field: timestamp)

# código de salida: 0
```

Figura 26. Sink de auditoría y tablas de Cloud Audit Logs en BigQuery

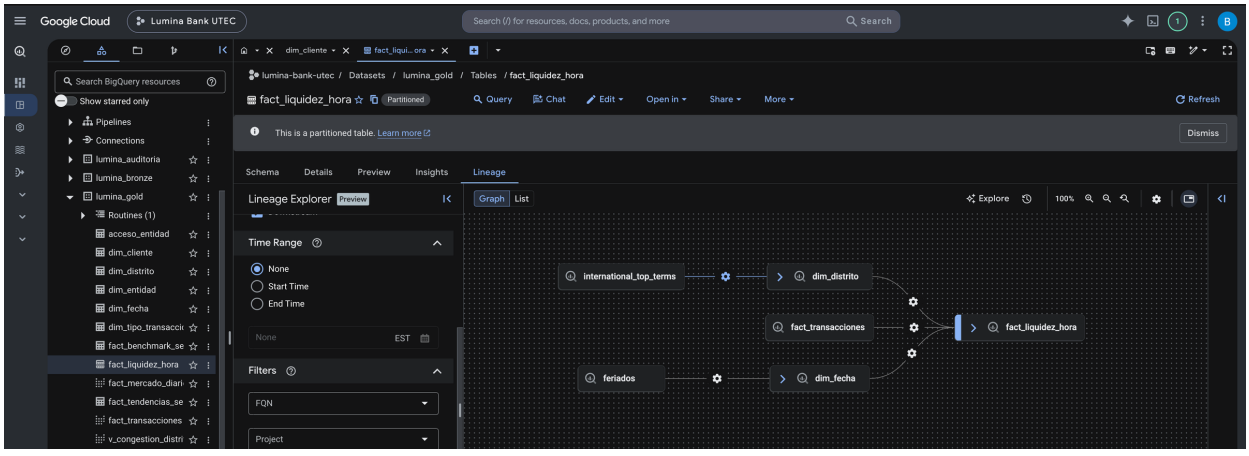


Figura 27. Linaje de `fact_liquidez_hora` en BigQuery: viene de `fact_transacciones`, `dim_distrito` (Google Trends) y `dim_fecha` (feriados). `dim_distrito` entra solo como la lista de los ocho distritos sobre la que se arma la serie por hora; Trends le da el nombre de región y no es variable del pronóstico. El tramo de Spark lo publica Dataproc; los pasos de BigQuery los registramos con eventos OpenLineage (`infra/registrar_linaje.py`)

6.3 Pipeline batch

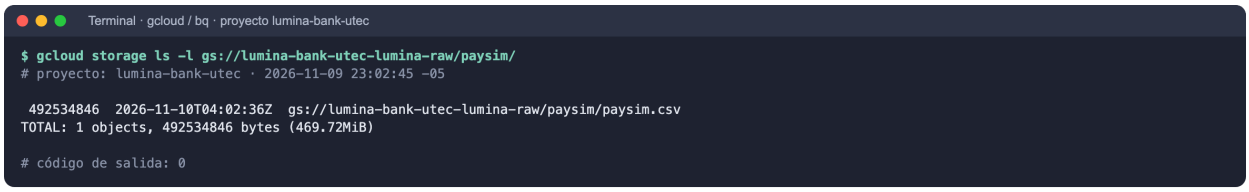


Figura 28. PaySim normalizado en la zona raw del Data Lake (469,72 MiB)

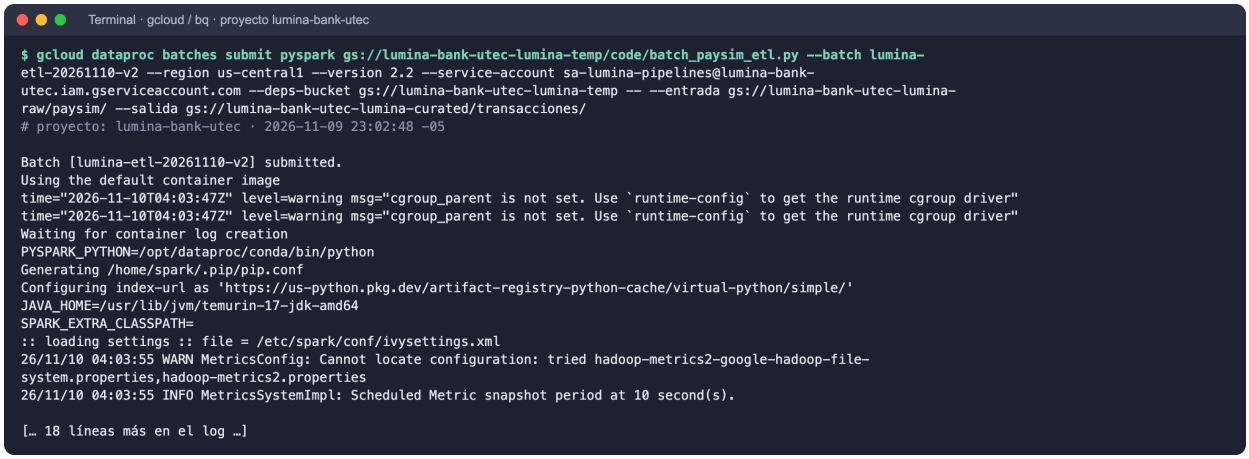


Figura 29. Envío del job de Spark a Dataproc Serverless con la línea de control de calidad



Figura 30. Estado final del job de Spark: SUCCEEDED en 2 min 51 s

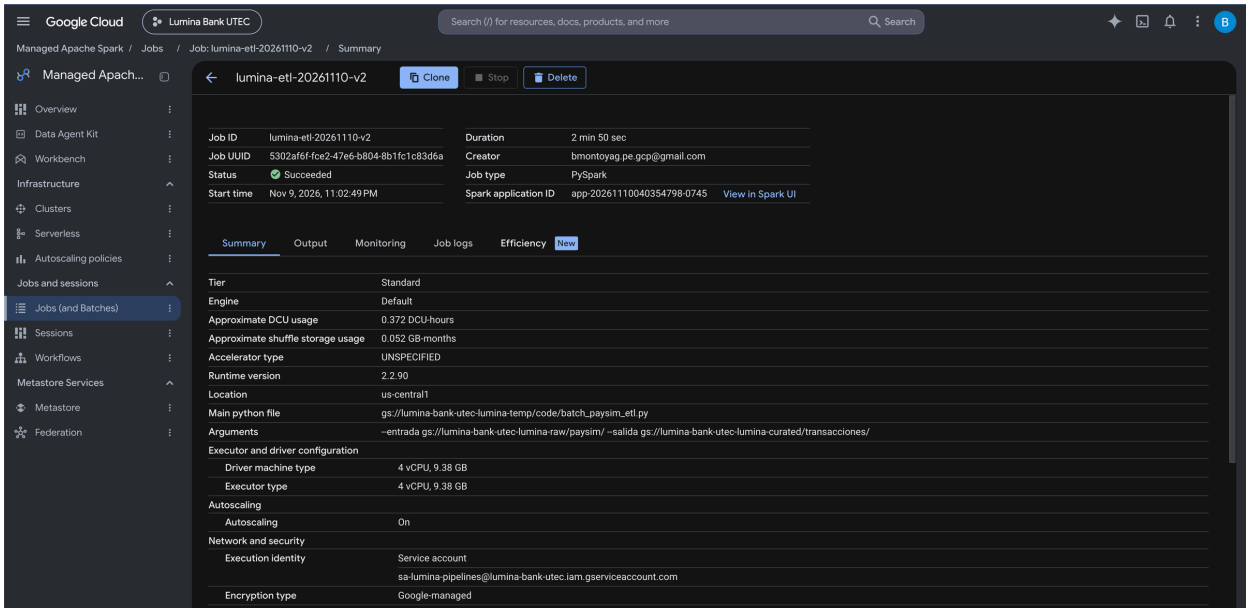


Figura 31. Consola de Dataproc Serverless, que la consola ya muestra como Managed Apache Spark: batch `lumina-etl-20261110-v2` terminado en 2 min 50 s con la cuenta de servicio de los pipelines (0,37 DCU-hora)



Figura 32. Zona curated: 31 particiones por fecha en Parquet

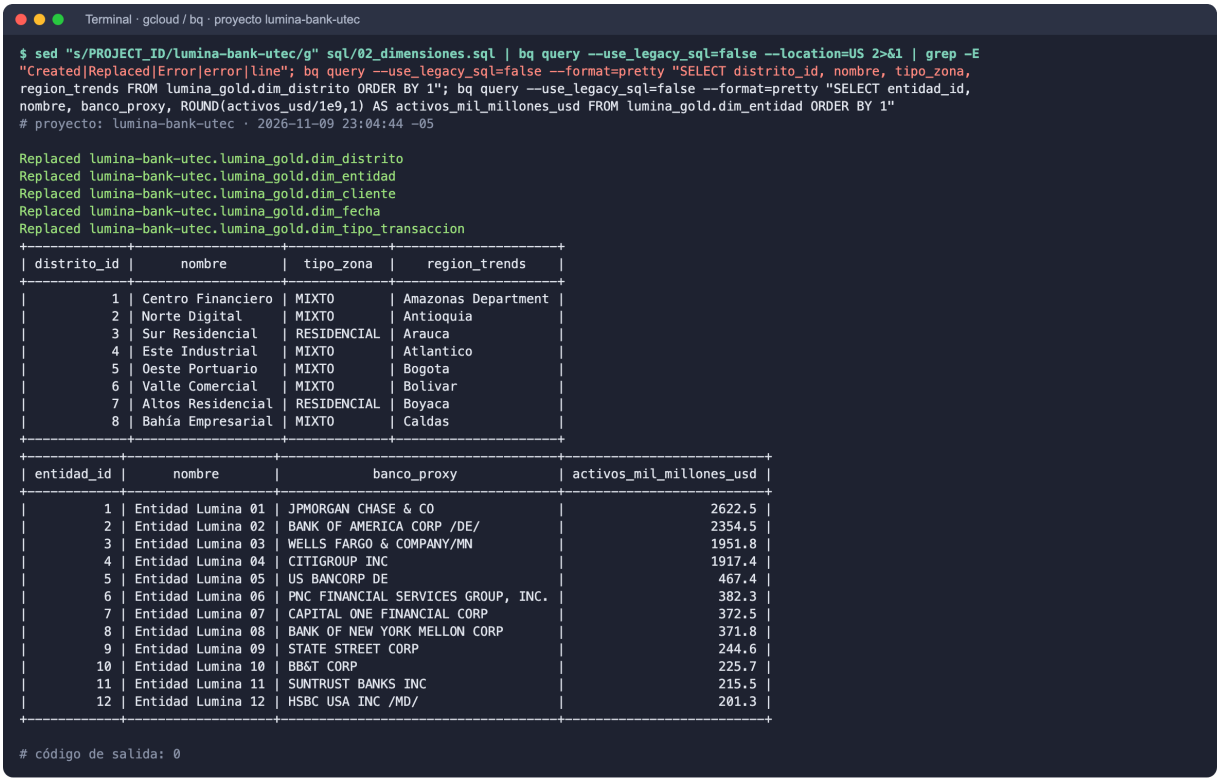


Figura 33. Dimensiones conformadas: distritos asociados a regiones de Trends y entidades con su banco de referencia en la SEC

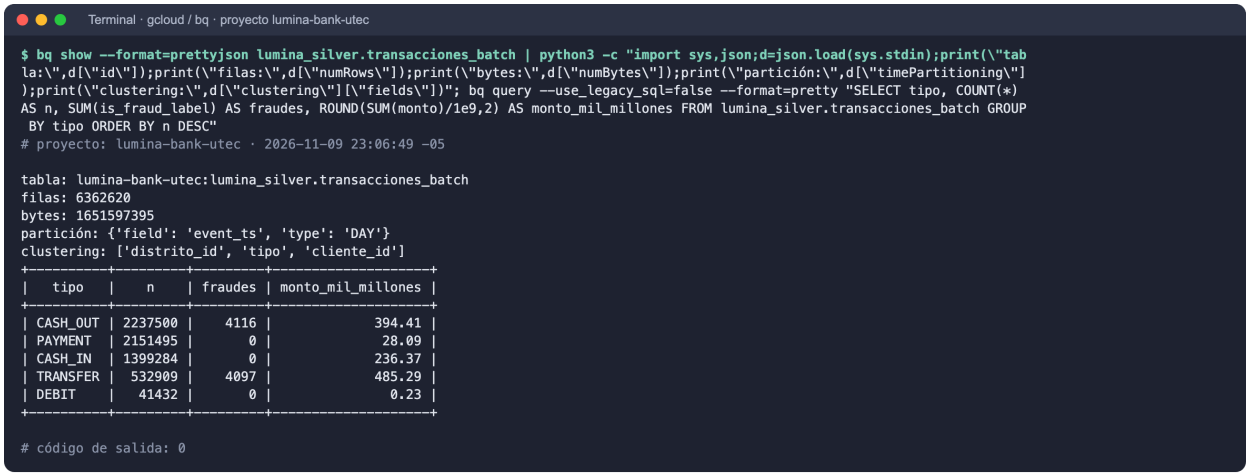


Figura 34. Tabla Silver de transacciones batch: 6 362 620 filas, particionada y agrupada

6.4 Pipeline streaming

Terminal · gcloud / bq · proyecto lumina-bank-utec

```
$ python publish_transactions.py --project lumina-bank-utec --csv paysim.csv --solo-transfer-cashout --rate 100 --limit 30000
# proyecto: lumina-bank-utec · 2026-11-09 23:07:27 -05

1000 eventos publicados (100 ev/s)
2000 eventos publicados (100 ev/s)
3000 eventos publicados (100 ev/s)
4000 eventos publicados (100 ev/s)
5000 eventos publicados (100 ev/s)
6000 eventos publicados (100 ev/s)
7000 eventos publicados (100 ev/s)
8000 eventos publicados (100 ev/s)
9000 eventos publicados (100 ev/s)
10000 eventos publicados (100 ev/s)
11000 eventos publicados (100 ev/s)

[… 19 líneas más en el log …]
```

Figura 35. Simulador publicando 30 000 transacciones a 100 por segundo

Terminal · gcloud / bq · proyecto lumina-bank-utec

```
$ bq query --use_legacy_sql=false --format=pretty 'SELECT decision, COUNT(*) AS n FROM lumina_silver.transacciones_stream WHERE
ingest_ts BETWEEN "2026-11-10 04:08:00" AND "2026-11-10 04:15:00" GROUP BY ROLLUP(decision) ORDER BY n' ; bq query
--use_legacy_sql=false --format=pretty 'SELECT origen, COUNT(*) AS filas, COUNT(DISTINCT event_id) AS event_id_distintos,
MIN(fecha) AS desde, MAX(fecha) AS hasta FROM lumina_gold.fact_transacciones GROUP BY origen' ; bq query --use_legacy_sql=false
--format=pretty 'SELECT COUNT(*) AS event_id_en_ambos_origenes FROM lumina_silver.transacciones_stream s JOIN
lumina_silver.transacciones_batch b USING (event_id)'
# proyecto: lumina-bank-utec · 2026-11-10 22:58:35 -05

+----------+-------+
| decision |   n   |
+----------+-------+
| BLOQUEAR |    50 |
| REVISAR  |    80 |
| APROBAR  | 29870 |
| NULL     | 30000 |
+----------+-------+


+--------+---------+--------------------+------------+------------+
| origen |  filas  | event_id_distintos |   desde    |   hasta    |
+--------+---------+--------------------+------------+------------+
| stream |  125274 |             125274 | 2026-11-10 | 2026-11-10 |
| batch  | 6362620 |            6362620 | 2026-07-01 | 2026-07-31 |
+--------+---------+--------------------+------------+------------+
+---------------------------+
| event_id_en_ambos_origenes |
+---------------------------+
|                         0 |
+---------------------------+

# código de salida: 0
```

Figura 36. Decisiones de las 30 000 transacciones de la prueba funcional y revisión de duplicados entre el histórico y el stream

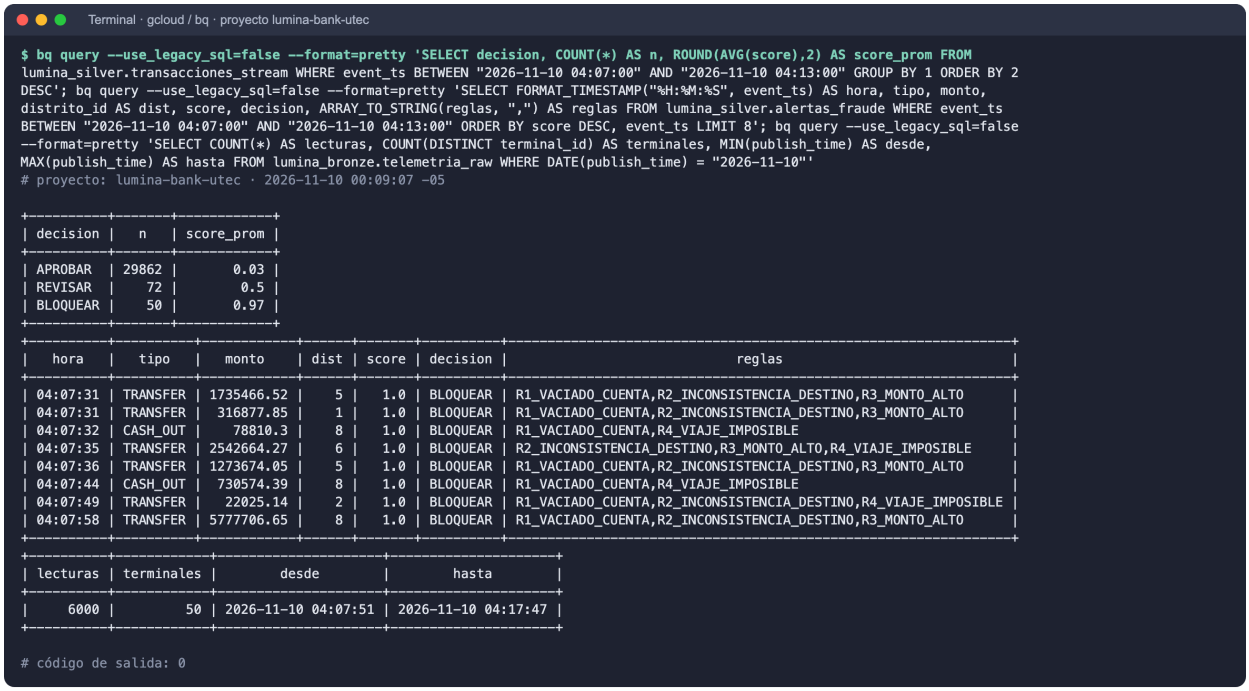


Figura 37. Decisiones del pipeline, alertas con las reglas que las dispararon y telemetría escrita en Bronze



Figura 38. Job `lumina-fraude-stream-v3` (corrida 3) con workers sin IP pública en la subred `lumina-datos-us-central1`. Lo lanzamos solo con `--subnetwork`; en ese caso la API deja `network: default` en el descriptor, pero la red la define la subred, que solo existe en `lumina-vpc` (figura 16)

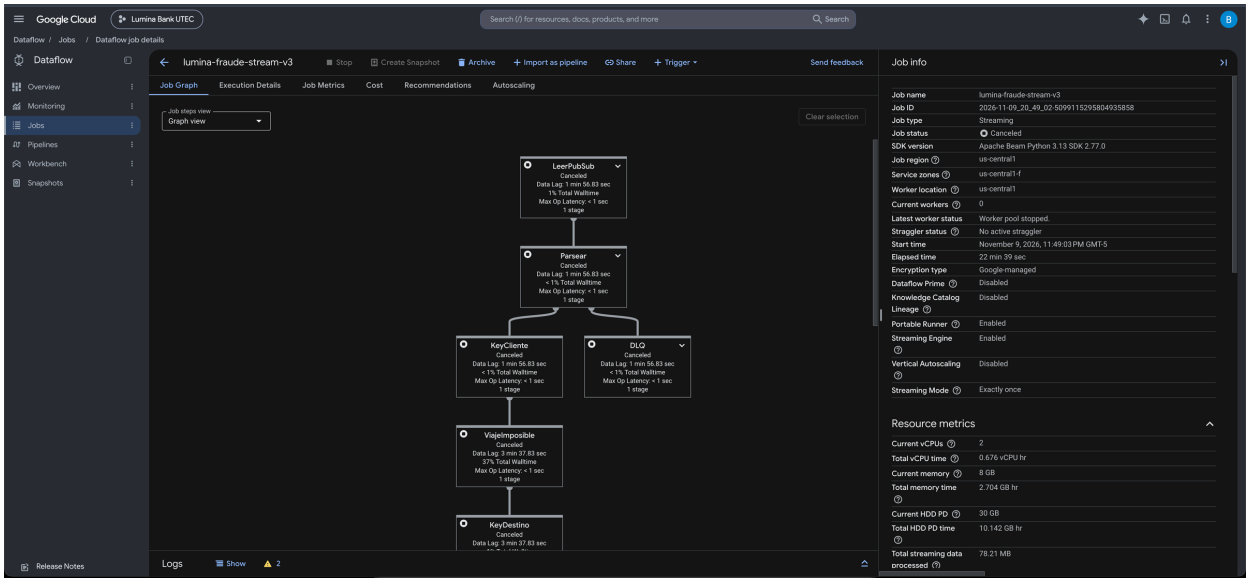


Figura 39. Grafo del job `lumina-fraude-stream-v3`: lectura de Pub/Sub, parseo con salida a DLQ y etapas con estado por cliente. `ViajeImposible` concentra el 37 % del tiempo de procesamiento

6.5 Pruebas de carga

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud dataflow jobs update-options 2026-11-09_20_05_40-16619663328803287800 --region us-central1 --min-num-workers 1 --max-
num-workers 1 --format='yaml(id,name,currentState,runtimeUpdatableParams)'
# proyecto: lumina-bank-utec · 2026-11-09 23:18:50 -05

currentState: JOB_STATE_RUNNING
id: 2026-11-09_20_05_40-16619663328803287800
name: lumina-fraude-stream
runtimeUpdatableParams:
  maxNumWorkers: 1
  minNumWorkers: 1
  workerUtilizationHint: 0.8

# código de salida: 0
```

Figura 40. Ajuste en caliente del rango de workers con `update-options`, el mismo mecanismo que usa el DAG de precalentamiento

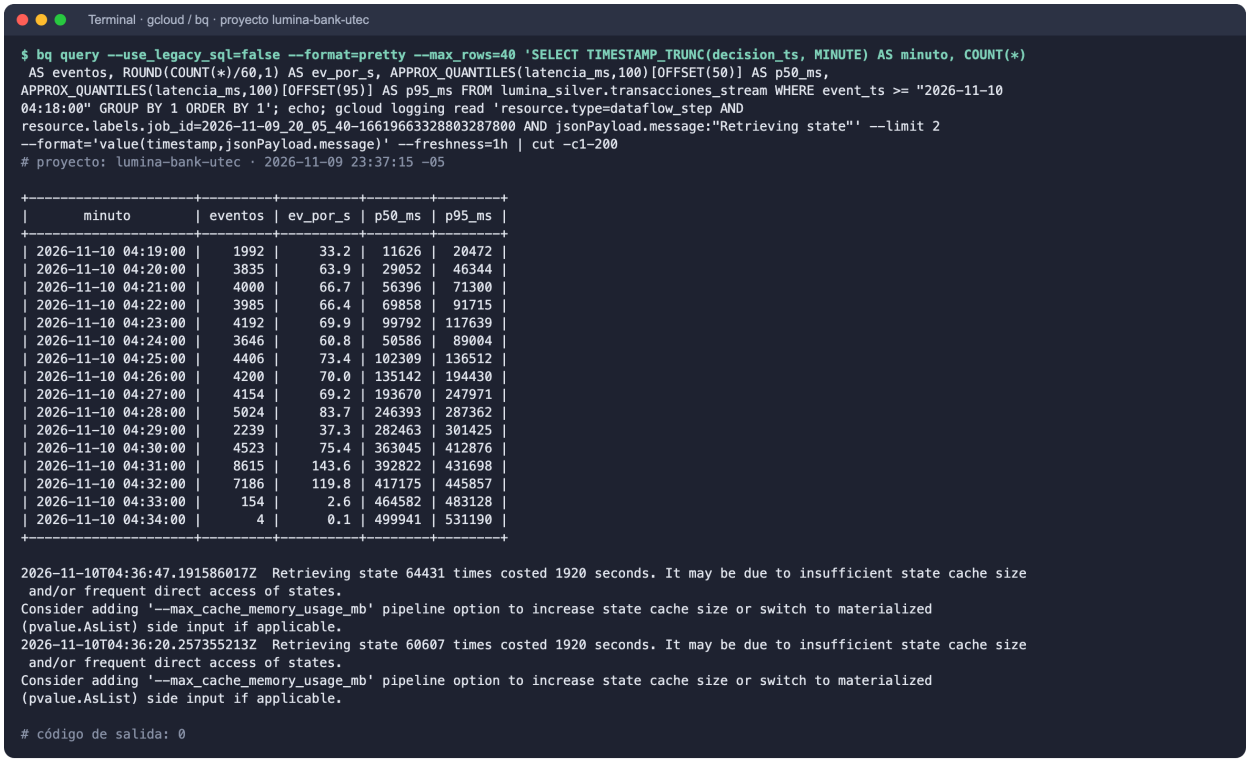


Figura 41. Corrida 1: eventos por minuto y latencia, con el aviso de lecturas de estado lentas

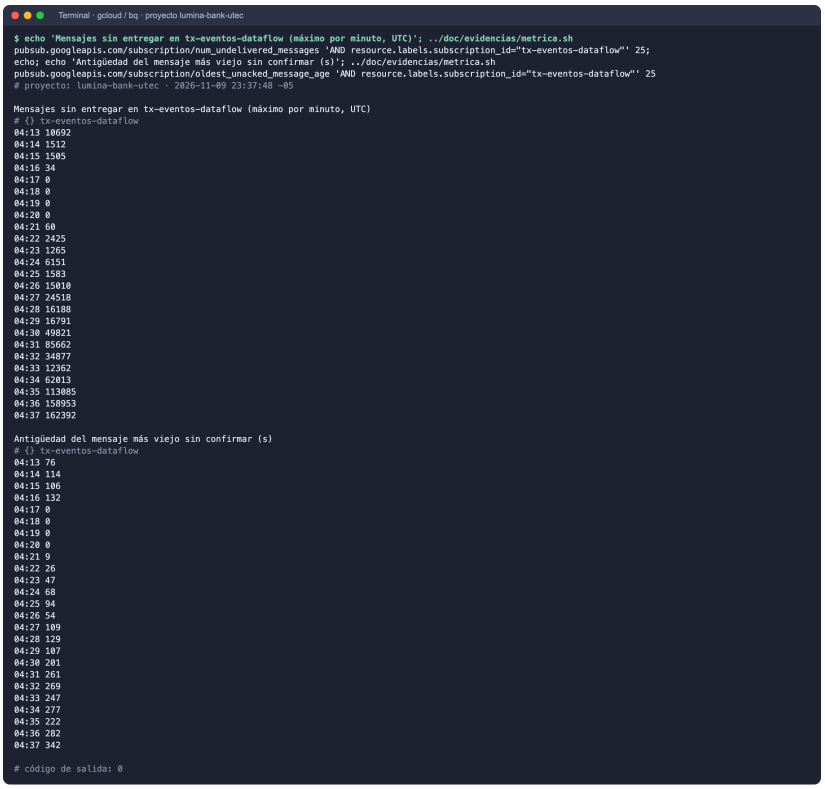


Figura 42. Corrida 1: backlog de la suscripción y antigüedad del mensaje más viejo

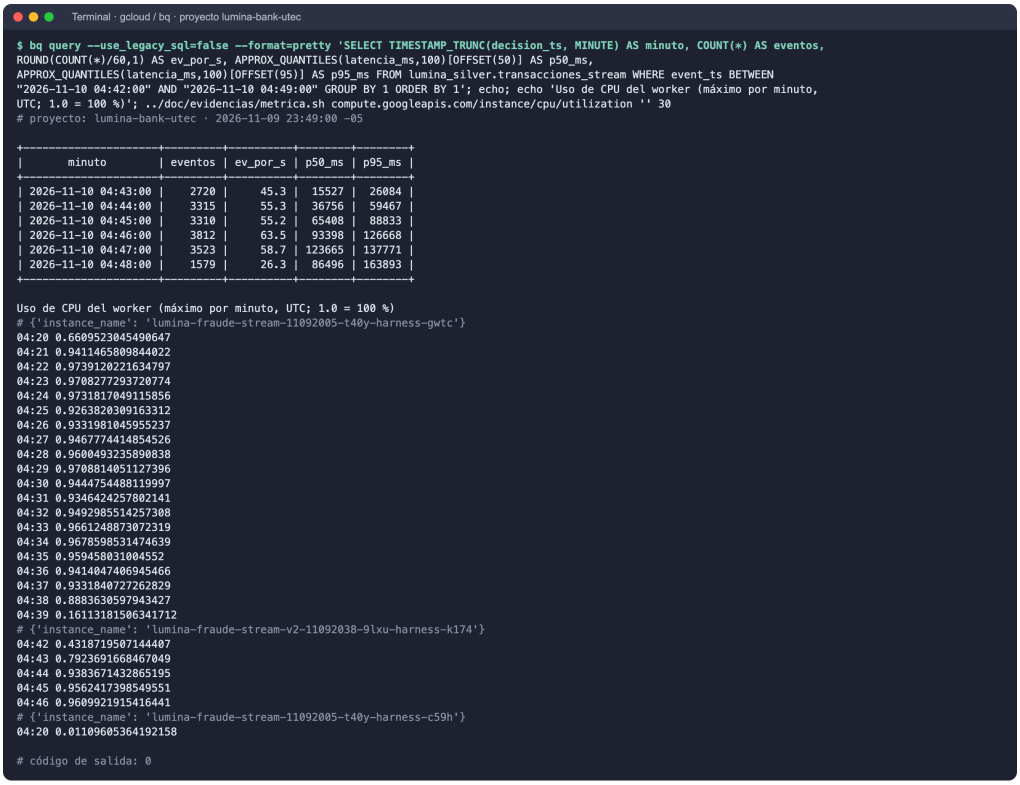


Figura 43. Corrida 2: eventos por minuto y uso de CPU del worker

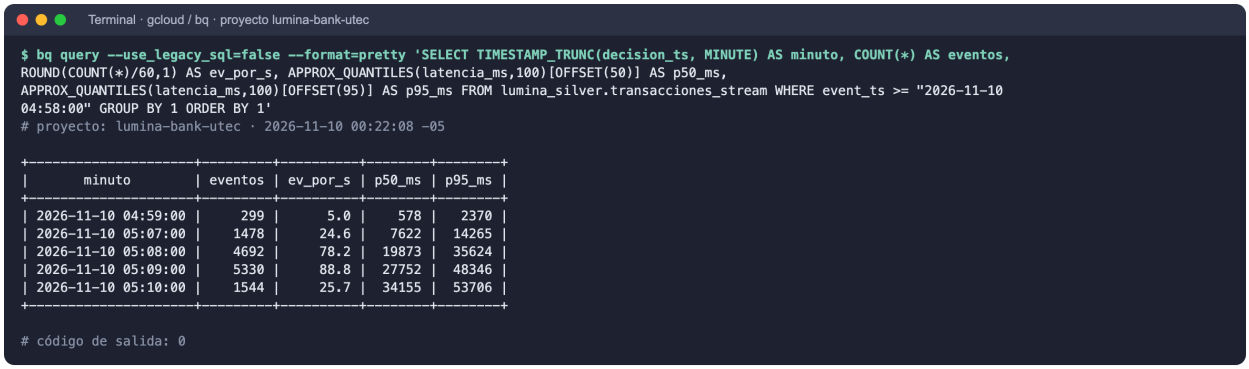


Figura 44. Corrida 3: eventos por minuto con la escritura a BigQuery en lotes. El primer minuto es la prueba de calentamiento a 5 ev/s, con p95 de 2,4 s

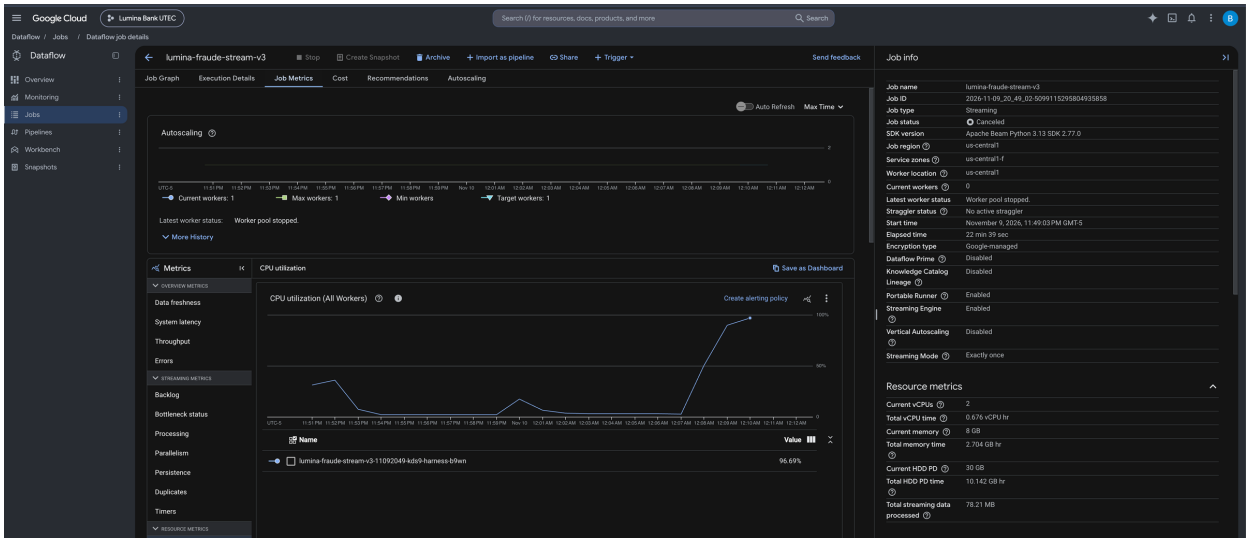


Figura 45. Consola, referencial: métricas del job `v3`: un solo worker (máximo 1) y la CPU sube a 97 % cuando entra la carga

6.6 Gold y BigQuery ML

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ sed "s/PROJECT_ID/lumina-bank-utec/g" sql/03_gold.sql | bq query --use_legacy_sql=false --location=US 2>&1 | grep -E
"^(Created|Replaced)|rror"
# proyecto: lumina-bank-utec · 2026-11-09 23:07:58 -05

Created lumina-bank-utec.lumina_gold.fact_transacciones
Created lumina-bank-utec.lumina_gold.fact_liquidez_hora
Created lumina-bank-utec.lumina_gold.fact_tendencias_semana
Created lumina-bank-utec.lumina_gold.fact_benchmark_sector
Created lumina-bank-utec.lumina_gold.fact_mercado_diario
Created lumina-bank-utec.lumina_gold.v_kpi_operativos_diario
Created lumina-bank-utec.lumina_gold.v_ingreso_por_producto
Created lumina-bank-utec.lumina_gold.v_desempeno_reglas_fraude
Created lumina-bank-utec.lumina_gold.v_salud_financiera_vs_sector
Created lumina-bank-utec.lumina_gold.v_congestion_distrito_hora
Created lumina-bank-utec.lumina_gold.v_salud_terminales

# código de salida: 0
```

Figura 46. Creación de los hechos y vistas de la capa Gold

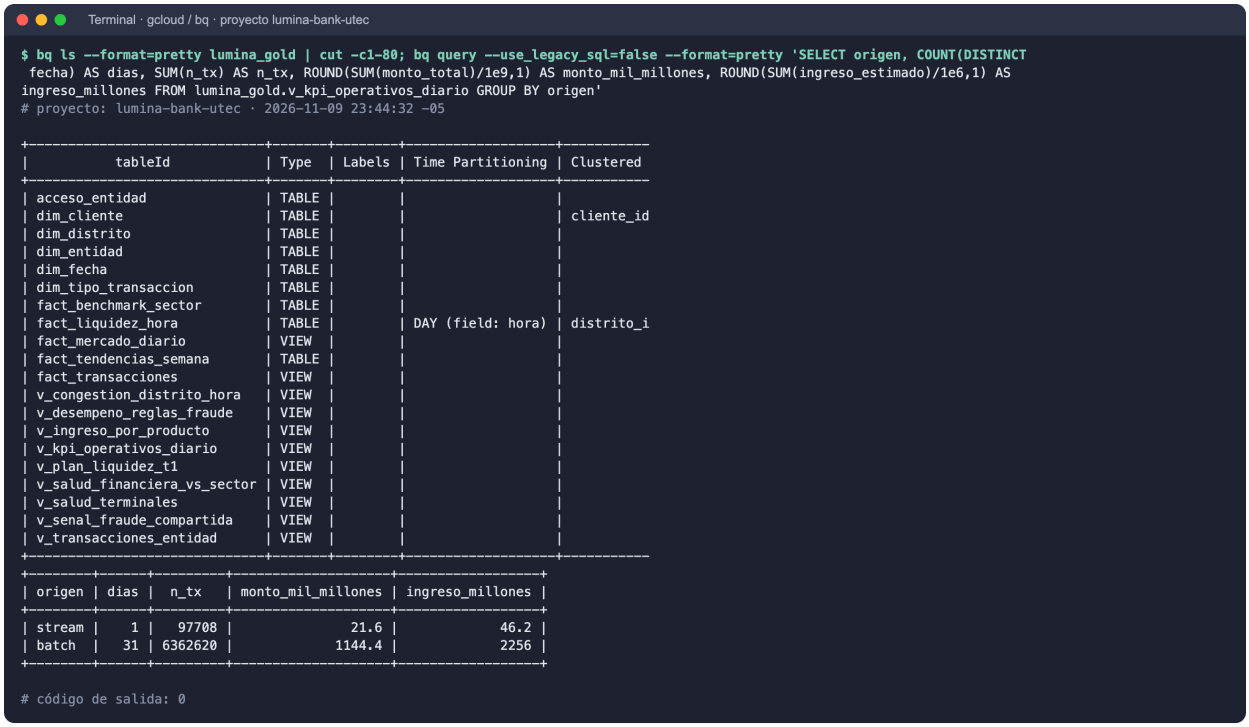


Figura 47. Objetos de Gold y KPI operativos por origen (batch y streaming)

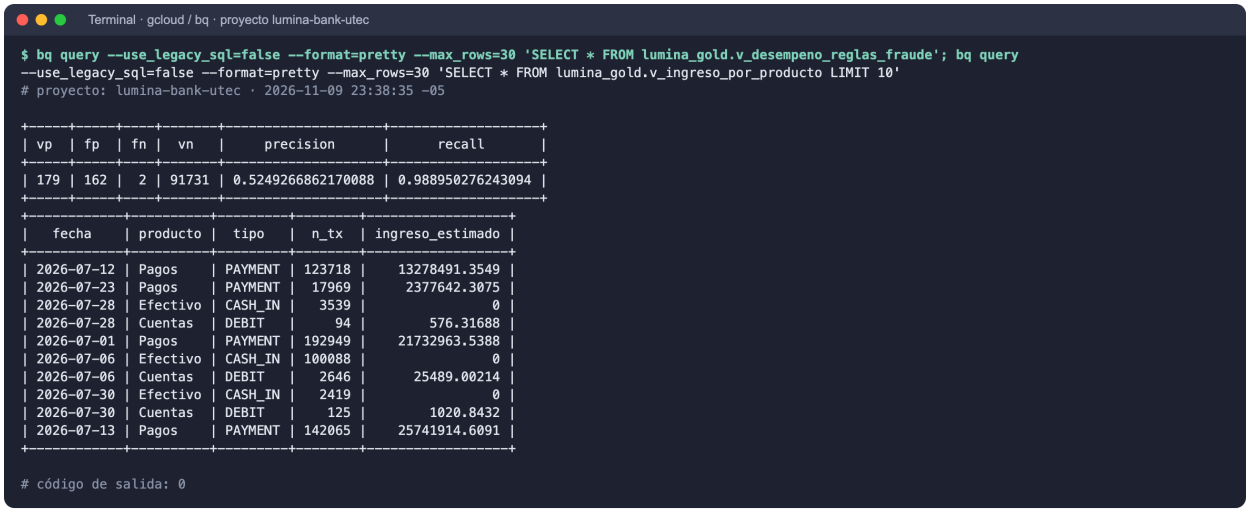


Figura 48. Desempeño de las reglas del pipeline e ingreso por producto

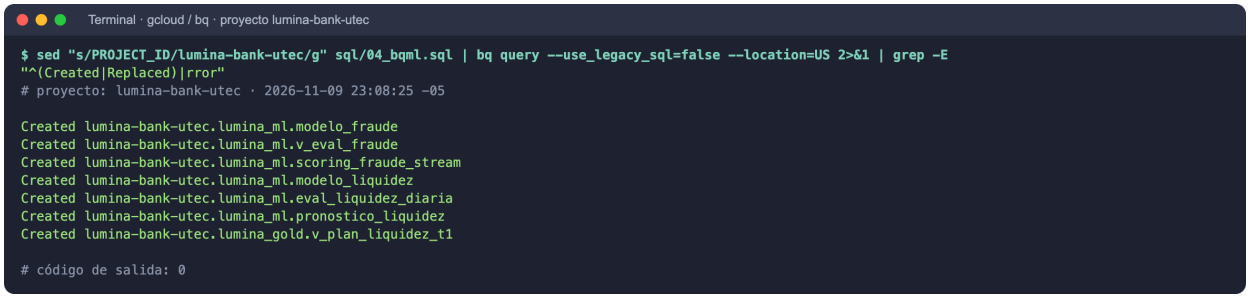


Figura 49. Entrenamiento de los modelos de fraude y liquidez

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bq query --use_legacy_sql=false --format=pretty --max_rows=30 'SELECT ROUND(precision,4) AS precision, ROUND(recall,4) AS
recall, ROUND(f1_score,4) AS f1, ROUND(roc_auc,4) AS roc_auc, ROUND(accuracy,4) AS accuracy FROM ML.EVALUATE(MODEL
lumina_ml.modelo_fraude, TABLE lumina_ml.v_eval_fraude, STRUCT(0.5 AS threshold))'; bq query --use_legacy_sql=false
--format=pretty --max_rows=30 'SELECT feature, ROUND(attribution,4) AS importancia FROM ML.GLOBAL_EXPLAIN(MODEL
lumina_ml.modelo_fraude) ORDER BY attribution DESC LIMIT 6'
# proyecto: lumina-bank-utec · 2026-11-09 23:38:20 -05

+-----------+--------+--------+---------+----------+
| precision | recall |   f1   | roc_auc | accuracy |
+-----------+--------+--------+---------+----------+
|    0.8107 | 0.9953 | 0.8935 |  0.9977 |   0.9993 |
+-----------+--------+--------+---------+----------+
+------------------+-------------+
|     feature      | importancia |
+------------------+-------------+
| err_saldo_orig   |      2.3113 |
| saldo_nuevo_orig |      0.4417 |
| saldo_ant_orig   |      0.1817 |
| saldo_ant_dest   |      0.1645 |
| monto            |       0.074 |
| hora_dia         |      0.0479 |
+------------------+-------------+

# código de salida: 0
```

Figura 50. Evaluación del modelo de fraude y variables más importantes

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ bq query --use_legacy_sql=false --format=pretty --max_rows=30 'SELECT * FROM lumina_ml.eval_liquidez_diaria ORDER BY 1 LIMIT
10'; bq query --use_legacy_sql=false --format=pretty --max_rows=30 'SELECT * FROM lumina_gold.v_plan_liquidez_t1 ORDER BY
distrito_id, fecha LIMIT 10'
# proyecto: lumina-bank-utec · 2026-11-09 23:38:25 -05

+-------------+------------+---------------------+---------------------+---------------------+
| distrito_id |   fecha    |     real_salidas    |      pronostico     |         ape         |
+-------------+------------+---------------------+---------------------+---------------------+
|           1 | 2026-07-28 |       1.1281021093E8 | 1.6526923093563554E8 |  0.4650201393399303 |
|           1 | 2026-07-29 |       3.9587781677E8 | 1.6850471592583546E8 |  0.5743517095737279 |
|           1 | 2026-07-30 |  9.834585046000001E7 |  1.692720838776221E8 |  0.7211919271212135 |
|           1 | 2026-07-31 | 1.7377066689999998E7 | 1.7003945182940876E8 |   8.785279349089544 |
|           2 | 2026-07-31 |        2.577196968E7 | 1.7035570533380654E8 |   5.610115852573304 |
|           2 | 2026-07-29 |        3.913911144E8 | 1.6883595113681626E8 |  0.5686259985854797 |
|           2 | 2026-07-30 |       1.1350569523E8 |  1.695817545922626E8 |  0.4940374070978023 |
|           2 | 2026-07-28 |        1.162906336E8 |  1.702460378848838E8 |  0.4639703354826661 |
|           3 | 2026-07-28 |       1.0066853415E8 | 5.8569280334835365E7 | 0.41819675006328316 |
|           3 | 2026-07-30 | 1.2338120174000001E8 |  6.180128275897135E7 |   0.499102927452396 |
+-------------+------------+---------------------+---------------------+---------------------+
+-------------+------------------+------------+-------------------+---------------------+
| distrito_id |     distrito     |   fecha    | efectivo_esperado | efectivo_recomendado |
+-------------+------------------+------------+-------------------+---------------------+
|           1 | Centro Financiero | 2026-07-28 |     1.6526923094E8 |      1.76392876705E9 |
|           1 | Centro Financiero | 2026-07-29 |     1.6850471593E8 |      3.09194456578E9 |
|           1 | Centro Financiero | 2026-07-30 |     1.6927208388E8 |      3.94989032369E9 |
|           1 | Centro Financiero | 2026-07-31 |     1.7003945183E8 |      4.64450028478E9 |
|           1 | Centro Financiero | 2026-08-01 |     1.7080681978E8 |      5.24450383701E9 |
|           1 | Centro Financiero | 2026-08-02 |     1.7157418773E8 |      5.78060630576E9 |
|           1 | Centro Financiero | 2026-08-03 |     1.7234155568E8 |      6.26976716273E9 |
|           1 | Centro Financiero | 2026-08-04 |      1.875594257E7 |       2.8284810731E8 |
|           2 | Norte Digital     | 2026-07-28 |     1.7024603788E8 |      1.22522293585E9 |
|           2 | Norte Digital     | 2026-07-29 |     1.6883595114E8 |      1.38345810817E9 |
+-------------+------------------+------------+-------------------+---------------------+

# código de salida: 0
```

Figura 51. Error del pronóstico por día y plan de liquidez por distrito, antes de corregir `v_plan_liquidez_t1`: el recomendado suma cotas horarias y por eso sale inflado

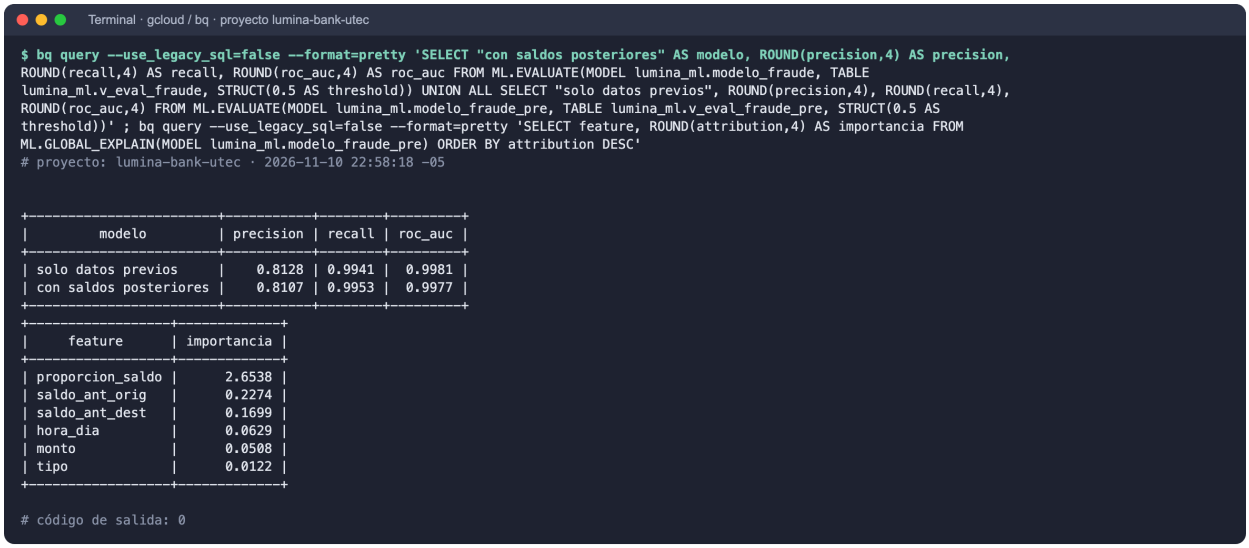


Figura 52. Modelo de fraude con y sin las variables posteriores a la transacción, y variables más importantes del modelo sin fuga



Figura 53. MAPE del pronóstico por distrito, con y sin el último día de PaySim

6.7 API de datos en Cloud Run

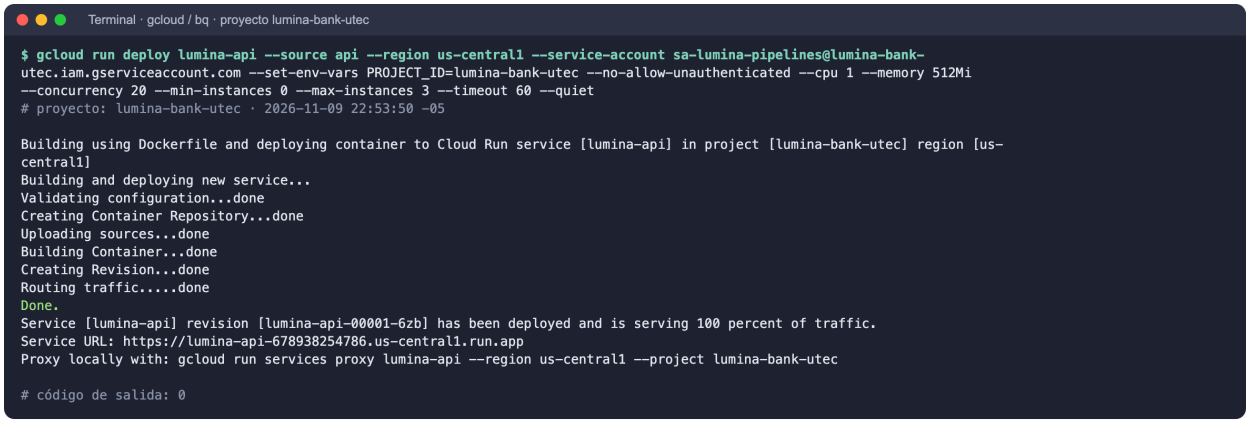


Figura 54. Despliegue de la API con concurrencia 20 y de 0 a 3 instancias

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ gcloud run services describe lumina-api --region us-central1 --format='yaml(metadata.name,status.url,spec.template.metadata.an
notations,spec.template.spec.containerConcurrency,spec.template.spec.timeoutSeconds,spec.template.spec.containers[0].resources.l
imits,spec.template.spec.serviceAccountName)'
# proyecto: lumina-bank-utec · 2026-11-09 23:39:58 -05

metadata:
  name: lumina-api
spec:
  template:
    metadata:
      annotations:
        autoscaling.knative.dev/maxScale: '3'
        run.googleapis.com/client-name: gcloud
        run.googleapis.com/client-version: 589.0.0
        run.googleapis.com/startup-cpu-boost: 'true'
    spec:
      containerConcurrency: 20
      containers:
      - resources:
          limits:
            cpu: '1'
            memory: 512Mi
      serviceAccountName: sa-lumina-pipelines@lumina-bank-utec.iam.gserviceaccount.com
      timeoutSeconds: 60
status:
  url: https://lumina-api-gk5x4vgopq-uc.a.run.app

# código de salida: 0
```

Figura 55. Configuración desplegada: máximo de 3 instancias, concurrencia 20, 1 vCPU y 512 MiB. La URL `lumina-api-gk5x4vgopq-uc.a.run.app` y la de la figura 54, basada en el número de proyecto, son dos alias que Cloud Run da a la misma revisión `lumina-api-00001-6zb`

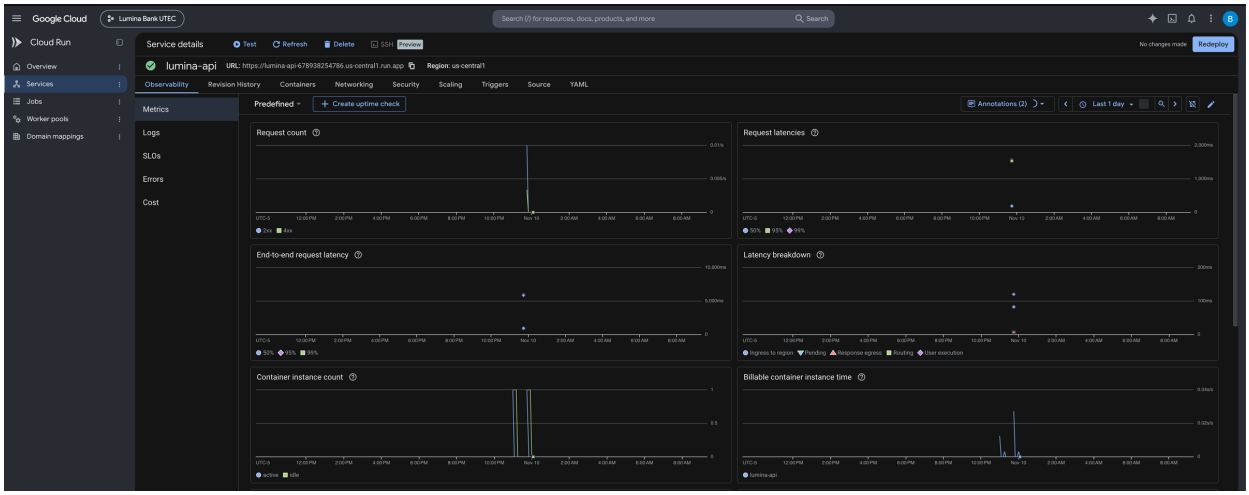


Figura 56. Métricas de `lumina-api` en Cloud Run: solicitudes 2xx y 4xx, latencia e instancias que vuelven a cero cuando no hay tráfico

```
Terminal · gcloud / bq · proyecto lumina-bank-utec

$ curl -s -o /dev/null -w 'HTTP %{http_code} sin credenciales
' https://lumina-api-678938254786.us-central1.run.app/salud
# proyecto: lumina-bank-utec · 2026-11-09 23:40:00 -05

HTTP 403 sin credenciales

# código de salida: 0
```

Figura 57. Sin credenciales la API responde 403

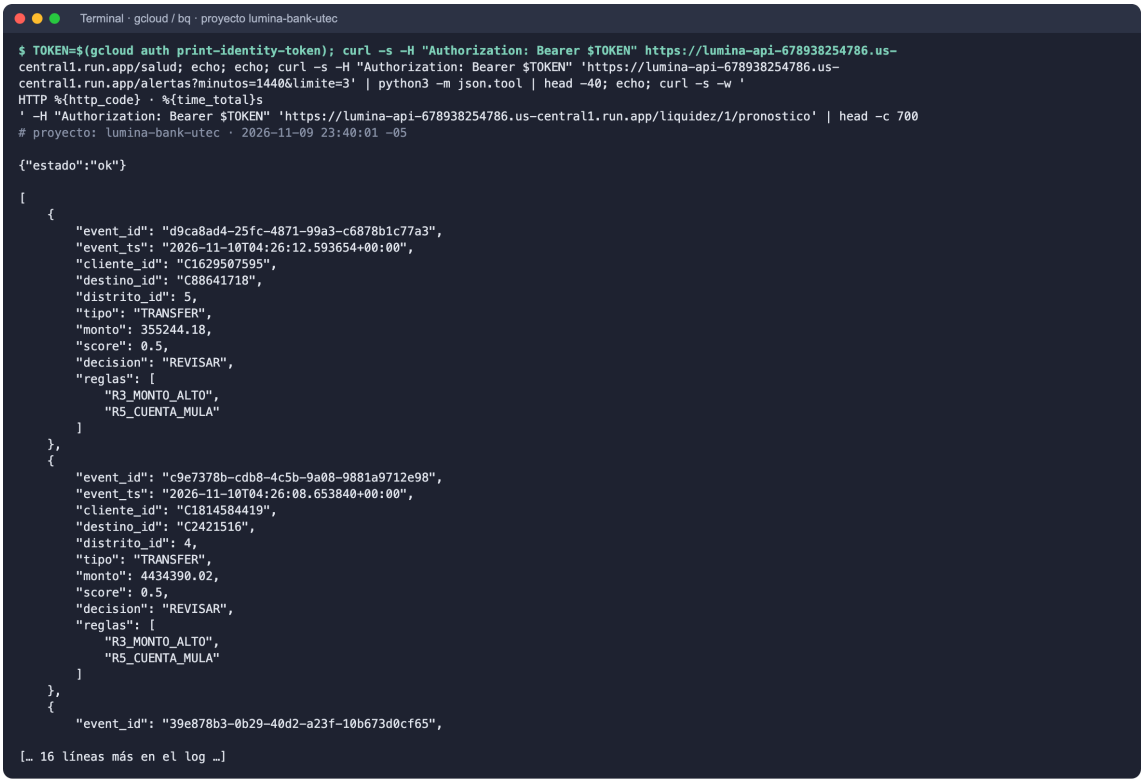


Figura 58. Con token, la API devuelve las alertas recientes y el pronóstico de liquidez

7 Pendientes para la Entrega 3

- Conectar Looker Studio a las vistas `v_kpi_*` y armar el tablero de la demo.
- Desplegar los dos DAG en Managed Service for Apache Airflow y mostrar una ejecución completa.
- Llevar el modelo de fraude al pipeline (hoy decide con reglas) y desplegar el servicio de autorización.
- Optimizar las etapas con estado del pipeline y repetir la prueba de carga hasta llegar a 500 eventos por segundo por worker.
- Repetir la prueba con autoscaling activo para confirmar que el job escala de 4 a 55 workers sin perder latencia y que la capacidad crece de forma lineal.
- Medir por separado el efecto de la VPC privada en la capacidad del worker (corrida de carga 2).
- Repetir la corrida de carga con `--max_cache_memory_usage_mb` más alto, como sugirió el runner, antes de concluir que el límite es solo la CPU.
- Dar a la API una cuenta de servicio propia, con lectura solo sobre las vistas seudonimizadas, y dejar de devolver `cliente_id` en claro.
- Separar `sa-lumina-pipelines` en una cuenta por componente, con roles sobre el dataset o bucket que usa.
- Extender la llave CMEK a Silver, Bronze y las zonas raw y curated.
- Asignar los policy tags a las columnas sensibles (`cliente_id` y saldos como `PII_ALTA`; `lat`, `lon` y `device_id` como `PII_MEDIA`) y mostrarlo con `bq show --schema`.

- Lanzar Dataproc Serverless en `lumina-vpc` con `--subnet lumina-datos-us-central1`.
- Crear el esquema de Pub/Sub de `tx-eventos` y dejar en Dataflow solo las reglas de dominio.
- Volver a capturar el plan de liquidez con la vista corregida.
- Poner el balanceador con Cloud Armor delante de la API y cerrar el acceso directo a su URL.
- Pasar la normalización de PaySim al ETL de Spark.
- Preparar la demo de punta a punta: simulador, Pub/Sub, Dataflow, alerta, API y tablero.

A Cambios en el repositorio

Archivo	Cambio
`doc/arquitectura/lumina.c4`	Modelo C4 en LikeC4; `render.sh` exporta las vistas
`doc/evidencias/`	Logs de cada comando y generador de capturas
`src/infra/gobierno_seguridad_red.sh`	VPC, firewall, KMS, auditoría, policy tags y Knowledge Catalog
`src/sql/05_gobierno_seguridad.sql`	Vistas por entidad y señal de fraude compartida
`src/sql/06_validacion_capacidad.sql`	Análisis de la prueba de carga
`src/orchestration/dag_lumina_precalentamiento.py`	Sube y baja el mínimo de workers en días de pago
`src/sql/02_dimensiones.sql`, `03_gold.sql`	Trends con Colombia como referencia y SEC con el año 2018
`src/infra/setup.sh`	Rol `pubsub.viewer` para la cuenta de los pipelines
`src/dataflow/fraud_streaming_pipeline.py`	Escritura a BigQuery en lotes y parámetros de ejecución medidos
`src/sql/04_bqml.sql`	Modelo de fraude sin variables posteriores a la transacción
`src/infra/registrar_linaje.py`	Registra el linaje de los pasos de BigQuery y del streaming